From Simulation to Hardware: Robust PI Voltage Control of a DC–DC Boost Converter by Metaheuristic Tuning, Gain Scaling and Low-Cost Hardware-in-the-Loop Optimization with an Arduino and MATLAB

Abstract

Proportional–integral control of step-up converters is tuned either in simulation, where sampling and quantization are idealized, or by trial and error on the bench. This paper links the two. Particle swarm optimization, grey wolf optimization, a genetic algorithm and a multi-objective genetic algorithm minimize a worst-case cost over operating scenarios, first for a 500-watt converter, where the optimum is integral-dominant because of the right-half-plane zero, and then for a laboratory-scale converter, in simulation and directly on hardware. The hardware uses a basic Arduino Uno and a MATLAB serial link, without dSPACE, STM32 or other costly real-time equipment. The best simulated gains were converted to Arduino units by an analytically derived factor of 6.23 and tested on the bench. Direct transfer failed: the scaled best simulation design overshot by 30 ± 12 percent, and its cost was 3.3 times that of the best hardware-tuned design and indistinguishable from the baseline gains. A simulation with the controller sampled at 1 millisecond raised the overshoot of the simulation optima from 4 to 88 percent, pointing to the sampling delay, but overestimated the bench overshoot by a factor of 2.5 to 5.8. Gains tuned on the converter shortened the rise time from 72 to 16–23 milliseconds and lowered the validation cost by 40 to 69 percent; their integral gains were three to six times lower than the simulation optimum. With three hardware runs per optimizer, no optimizer could be ranked statistically.

Keywords / Index Terms: *Arduino, boost converter, gain scaling, hardware-in-the-loop optimization, particle swarm optimization, PI control, simulation-to-hardware gap.*

1 Introduction

DC–DC boost converters raise a low input voltage to a regulated higher output voltage and are a basic building block of photovoltaic systems, fuel-cell interfaces, DC microgrids and battery-based supplies [1–3]. At the primary control level, voltage-mode proportional–integral (PI) control remains the most common choice because of its simple structure, low computational burden and ease of implementation [3–5].

Choosing the PI gains is nevertheless less straightforward than it appears. Classical rules such as Ziegler–Nichols [6], the procedures collected in [7] and the automatic tuners compared in [8] are normally applied to a single nominal plant, whereas the small-signal control-to-output transfer function of a boost converter depends on the operating point and is non-minimum phase, with a right-half-plane (RHP) zero in continuous conduction [1, 9]. An RHP zero cannot be removed by feedback and limits the achievable bandwidth to a fraction of the zero frequency [10], while the output filter introduces a resonance whose damping changes with load [1]. When the inductor current becomes discontinuous at light load, the dynamics change again and the continuous-conduction model no longer applies [1, 11]. A controller tuned for the nominal point can therefore become lightly damped or unstable away from it. Better-performing structures exist, including current-mode control [12], nonlinear and flatness-based designs [13], model predictive control and reinforcement learning [14, 15], including an adaptive robust PI controller whose initial gains are set by the snake optimization algorithm and adapted by reinforcement learning for the RHP-zero plant [16] and a

finite-control-set predictive controller combined with a PI loop to mitigate the non-minimum-phase response [17], but they add modelling effort, sensing or computing requirements, so a well-tuned PI controller remains attractive and deserves a rigorous design procedure.

Metaheuristic optimization has been widely used to tune PI/PID controllers. Particle swarm optimization (PSO) [18,19] was applied early to PID design for voltage regulation [20] and has since been used for DC–DC converters in several forms: PSO as a direct controller for a boost converter implemented on a microcontroller [21], hybrid PSO–GSA tuning of PI and sliding-mode controllers for a Ćuk converter [22], PSO-optimized PI control of a buck converter emulating a fuel cell [23], and PSO versus Ziegler–Nichols tuning in a PV system containing a boost stage [2]. Comparative studies of PSO, GA and GWO for the PID control of a boost converter report that GWO gave the lowest RMSE [24], the whale optimization algorithm [25] has been used for PID design of a buck converter [26], and GWO has been used to tune an indirect IMC-PID controller for a boost converter while accounting explicitly for its RHP zero [27]. Other metaheuristics have also been used, for example the artificial bee colony algorithm for fractional-order PID control of a boost converter modelled with its non-minimum-phase dynamics [28], and a recent study reports experimental validation of PSO- and LPO-tuned PI gains on a boost converter prototype [3]. Multi-objective stochastic optimization has also been used for the design of DC–DC converters [29]. Grey wolf optimization (GWO) [30], genetic algorithms (GA) [31] and NSGA-II [32] are standard benchmarks, and broader surveys are available in [33]. Optimizers have also been run *online*: a DSP-based PSO was used for the online optimization of power system stabilizers [34], and an online PSO was used to auto-tune the PI controller of a dual active bridge converter [35]. Table 1 summarizes selected related studies.

A second strand of the literature concerns the *platform* on which the controller is implemented. Rapid-prototyping systems such as dSPACE and OPAL-RT are widely used for converter control, but they rely on closed, licensed software and cost from several thousand to several tens of thousands of US dollars [36, 38, 39], which is why low-cost real-time platforms have been proposed [36, 40]. Open hardware such as Arduino boards is attractive for education and laboratory work [41], and has been used to drive boost and other DC–DC converters [37]. Digital implementation brings its own issues—sampling delay, quantization of the ADC and of the PWM duty, and the discretization of the control law [42–44]—and these are exactly the effects that an idealized simulation omits. The mismatch between idealized simulation and physical reality, the "reality gap", is well documented in robotics [45, 46], and a common remedy is to tune directly on the physical system with a measured cost [47, 48]. For power converters, however, studies that tune a PI controller in simulation, convert the gains to the integer arithmetic of a low-cost microcontroller, and then quantify how well those gains actually perform on the bench are scarce, and the scaling between simulation units and ADC/PWM counts is usually not documented.

Four limitations in this body of work motivate the present study. First, the tuning cost is often evaluated at a single operating point, with robustness checked afterwards by simulating the tuned controller under variable load and input voltage [24], so the resulting gains are not guaranteed to remain well behaved when the line, load or component values change. Second, stability margins are rarely used as explicit constraints in the optimization, and the comparison with a nominal-point automatic tuner is seldom made across several scenarios. Third, because metaheuristics are stochastic and, according to the no-free-lunch theorems, no algorithm is superior on all problems [49], conclusions about an optimizer require repeated runs and statistical testing, which is not always reported; moreover, the community has called for caution about metaphor-based algorithms and for rigorous benchmarking [50]. Fourth, the transfer of simulation-optimal gains to an inexpensive embedded implementation, and the size of the resulting performance gap, are rarely reported quantitatively.

The contributions of this paper are as follows.

1. A robustness-aware tuning framework for the PI voltage controller of a boost converter, in which a cost combining normalized error integral, overshoot, settling time and (in the design-level study) duty-cycle effort and stability-margin penalties is aggregated over several line, load and parameter

Table 1: Selected related studies on optimization-based, learning-based and low-cost implementations of control for DC–DC converters (summarized from the cited works).

Ref.	System	Method	Focus of the study
[21]	Boost converter	PSO used as the controller itself, standard and GLBest variants	Compared with a PI controller with anti-windup under different operating conditions; microcontroller implementation
[4]	Boost converter	Double-loop PI with an analytical parameter calculation procedure	Controller parameter calculation approach
[22]	Ćuk converter	Hybrid PSO–GSA tuning of PI and sliding-mode controllers	Controller design and analysis
[23]	Buck converter, fuel-cell emulator	PSO-optimized PI	MPPT testing and battery-charger application
[2]	PV system with boost stage	PSO versus Ziegler–Nichols tuning of PI controllers	Grid-connected PV system performance
[28]	Boost converter (non-minimum-phase model)	ABC-optimized PID and fractional-order PID	Transient response, control effort, robustness to input voltage
[29]	DC–DC converter design	Multi-objective stochastic optimization (GA, PSO, SA)	Power-stage design with Pareto fronts
[14]	Boost converter	Reinforcement-learning-based predictive control	Simulation and experiments, load fluctuations
[15]	Boost converter	PPO reinforcement learning versus optimized PI and ANN	Step-response comparison
[3]	Boost converter prototype	PSO and LPO PI gains versus root-locus PI	Experimental validation under input-voltage and load disturbances
[24]	Boost converter	PSO, GA and GWO tuning of a PID controller	RMSE comparison; response under variable load and input voltage
[26]	Buck converter	Whale optimization algorithm for PID design	Transient response, compared with GA
[27]	Boost converter (RHP zero)	GWO-tuned indirect IMC-PID controller	Comparison with PID and IMC-PID using IAE, ISE, ITAE and ITSE
[16]	Boost converter (RHP zero)	Adaptive robust PI: reinforcement learning with snake-optimization initial gains	Robustness to disturbances and parameter variations
[17]	Boost converter (non-minimum phase)	Finite-control-set MPC with a PI reference-current loop	Mitigation of the initial inverse response
[35]	Dual active bridge converter	Online PSO auto-tuning of the PI controller	Accuracy and speed of the converter control
[36]	Boost converter prototype	Discrete controllers implemented on a low-cost platform	Cost comparison with commercial rapid-prototyping platforms; simulation versus experiment
[37]	Buck, boost and buck–boost converters	Arduino Nano generating the switching signal	Design and simulation for low- and medium-power applications

scenarios (worst case plus mean), and in which PSO, GWO, GA and NSGA-II are compared over repeated independent runs with non-parametric statistical tests.

2. A design-level study of a 500 W converter that explains, through the coincidence of the RHP zero and the LC resonance, why the robust optimum is integral-dominant, and that shows how fragile manually and automatically tuned nominal-point designs are.

3. A complete, reproducible simulation setup of a laboratory-scale converter (corner scenarios, Monte Carlo analysis, disturbance tests, switching-model validation) and a matching hardware setup based on a basic Arduino Uno and the MATLAB serial-communication interface, with the analytically derived scaling factor that converts simulation gains to Arduino units.

4. A hardware-in-the-loop optimization in which the four optimizers tune the PI gains directly on the physical converter through the serial link, without dSPACE, STM32 or any other costly

Table 2: Overview of the studies reported in this paper.

	Study A	**Study B, simulation**	**Study B, hardware**
Converter	500 W, 12 V → 24 V	10 V → 18.2 V, 50 Ω	7.2 V supply, 8–15 V, 50 Ω
Switching frequency	10 kHz	62.5 kHz	62.5 kHz (Timer1 PWM)
Inductance, capacitance	1.44 mH, 4.34 mF	22.5 µH, 470 µF	same as simulation
Controller	Continuous PI	Continuous PI (Simulink)	Discrete PI, 1 kHz, Arduino Uno
Optimization model	Averaged, then switching re-rank	Averaged (CCM)	Measured on the converter
Scenarios / tests	7 scenarios	17 corners, 500 Monte Carlo	Steps 8→12→15→12→8 V
Optimizers	PSO, GWO, GA, NSGA-II	PSO, GA, GWO, NSGA-II	PSO, GA, GWO, NSGA-II
Independent runs	30 (two search spaces)	15	3
Evaluations per run	≤ 240 (averaged model)	680 (averaged model)	360 (measured)

real-time platform, together with a noise-aware metric set and repeated validation.

5. A quantitative, repeated bench evaluation of the best simulation gains after scaling, compared with the hardware-optimized gains and with derated versions, supplemented by switching simulations with a sampled controller at the bench input voltage; it exposes the simulation-to-hardware gap, relates it to the sampling delay, the conduction mode and the converter resolution, and yields practical guidelines.

The remainder of the paper is organized as follows. Section 2 describes the two converters and their models. Section 3 presents the overall framework and the methodology of the simulation studies, and Section 4 the simulation setup of the laboratory-scale converter. Section 5 describes the hardware platform, the gain scaling and the hardware test protocol. Sections 6–9 report the results of Study A, of the simulation of Study B, of the hardware optimization and of the transfer of the simulation gains to the hardware. Section 10 discusses the findings and Section 11 concludes.

2 Converters and Models

Two converters are studied. Study A is a 500 W design-level converter used to understand the structure of robust PI tuning on a boost power stage; Study B is a laboratory-scale converter that exists as a Simulink model and as a physical prototype. Table 2 summarizes the differences. Sections 2.1–2.4 describe Study A, Sections 2.5 and 2.6 Study B.

2.1 Study A: power stage

Study A uses a 500 W boost converter operating in continuous conduction mode (CCM) with an input voltage $V_{\text{in}} = 12\,\text{V}$, an output voltage $V_{\text{o}} = 24\,\text{V}$ and a switching frequency $f_{\text{s}} = 10\,\text{kHz}$. With the duty ratio $D = 1 - V_{\text{in}}/V_{\text{o}} = 0.5$, the inductance, capacitance and load resistance follow from the usual ripple-based design equations [1]:

$$L = \frac{V_{\text{in}}\,(V_{\text{o}} - V_{\text{in}})}{\Delta I_L\, f_{\text{s}}\, V_{\text{o}}}, \qquad C = \frac{I_{\text{o}}\,(1 - V_{\text{in}}/V_{\text{o}})}{f_{\text{s}}\,\Delta V_{\text{o}}}, \qquad R = \frac{V_{\text{o}}}{I_{\text{o}}} \tag{1}$$

where $I_{\text{o}} = P/V_{\text{o}}$ is the rated output current, $\Delta I_L = 0.01\ I_{\text{o}}\ (V_{\text{o}}/V_{\text{in}})$ is the inductor-current ripple and $\Delta V_{\text{o}} = 0.01\ V_{\text{o}}$ is the output-voltage ripple. The resulting values are listed in Table 3.

2.2 Study A: switching-level Simulink model

The converter was built in MATLAB/Simulink R2024a with a DC source, inductor, IGBT switch with gate input, diode, output capacitor and resistive load, and a PWM generator driven by the controller (Fig. 1). The IGBT has on-state resistance $R_{\text{on}} = 1\,\text{m}\Omega$ and forward voltage $V_{\text{f}} = 1\,\text{V}$, and the diode has $R_{\text{on}} = 1\,\text{m}\Omega$ and $V_{\text{f}} = 0.8\,\text{V}$; the internal inductance of both devices is set to zero. The diode uses a snubber of 500 Ω in series with 250 nF, and the IGBT snubber is disabled.

Table 3: Converter specification and component values.

Parameter	Symbol	Value
Rated power	P	500 W
Input voltage	V_{in}	12 V
Output voltage reference	$V_{\text{ref}} = V_{\text{o}}$	24 V
Switching frequency	f_{s}	10 kHz
Duty ratio (ideal)	D	0.50
Rated output current	I_{o}	20.83 A
Inductor current ripple	ΔI_L	0.417 A
Output voltage ripple	ΔV_{o}	0.24 V
Inductance	L	1.44 mH
Capacitance	C	4.34 mF
Load resistance	R	1.152 Ω

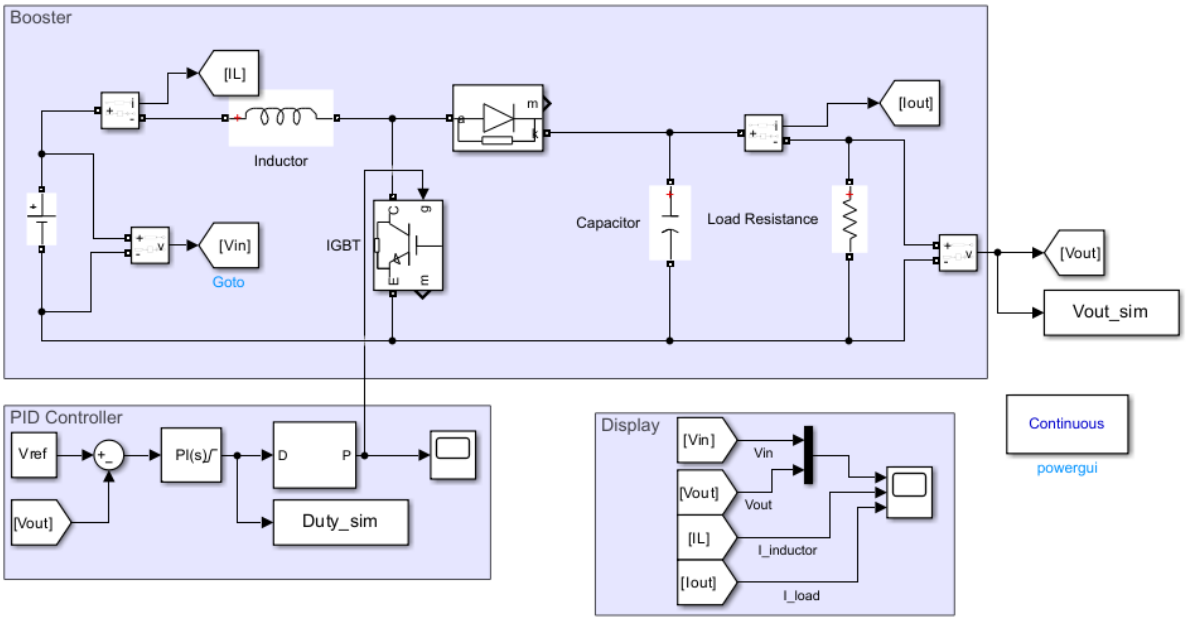


Figure 1: Simulink model of the closed-loop boost converter: power stage (source, inductor, switch, diode, capacitor and load), PI controller with PWM generator, and logging blocks.

The PWM carrier frequency is $f_{\text{sw}} = 10\,\text{kHz}$, and the PWM generator is run in continuous mode (sample time of zero). The controller is a parallel-form PI block, $C(s) = K_{\text{p}} + K_{\text{i}}/s$, acting on the error between the 24 V reference and the measured output voltage; its output is the duty ratio. The output-voltage and duty-ratio signals are logged for post-processing. The PI block output is limited to the range 0–0.9 and uses clamping anti-windup, so that the duty ratio stays within physical limits and the integrator does not wind up during start-up. The model is solved with a variable-step solver (automatic solver selection, default tolerances), and the power-electronics blocks are simulated with a continuous `powergui`. All time-domain results of Study A in Section 6 were obtained from this switching model.

2.3 Averaged small-signal model (continuous conduction)

For analysis, the margin computation and the fast optimization stage, the converter is represented by the standard CCM averaged small-signal model [1, 9, 51]. The control-to-output transfer function is

Table 4: Operating scenarios and averaged-model characteristics. † marks scenarios used inside the optimization cost.

Scenario	**Perturbation**	**D**	G_0 **(V)**	f_z **(Hz)**	f_0 **(Hz)**	Q
Nominal †	none	0.50	48.0	31.8	31.8	1.00
V_{in} − 20 % †	$V_{\mathrm{in}} = 9.6\,\mathrm{V}$	0.60	60.0	20.4	25.5	0.80
V_{in} + 20 %	$V_{\mathrm{in}} = 14.4\,\mathrm{V}$	0.40	40.0	45.8	38.2	1.20
Load 50 %	$R = 2.304\ \Omega$	0.50	48.0	63.7	31.8	2.00
Load 20 % †	$R = 5.760\ \Omega$	0.50	48.0	159.2	31.8	5.00
L − 20 %	$L = 1.152\,\mathrm{mH}$	0.50	48.0	39.8	35.6	1.12
C − 20 %	$C = 3.472\,\mathrm{mF}$	0.50	48.0	31.8	35.6	0.89

$$G_{vd}(s) = \frac{V_{\mathrm{in}}}{D'^2}\,\frac{1 - \dfrac{sL}{RD'^2}}{1 + \dfrac{sL}{RD'^2} + \dfrac{s^2LC}{D'^2}}, \qquad D' = 1 - D \tag{2}$$

which has a DC gain $G_0 = V_{\mathrm{in}}/D'^2$, an RHP zero at f_z and a resonant pair at f_0 with quality factor Q:

$$f_z = \frac{RD'^2}{2\pi L}, \qquad f_0 = \frac{D'}{2\pi\sqrt{LC}}, \qquad Q = D'R\sqrt{C/L} \tag{3}$$

With the PI controller

$$C(s) = K_{\mathrm{p}} + \frac{K_{\mathrm{i}}}{s} \tag{4}$$

the loop gain is L(s) = C(s) $G_{vd}(s)$, and gain and phase margins are computed from it (unity PWM gain and unity sensor gain are assumed). Because the plant is flat at G_0 up to the resonance and the RHP zero adds phase lag, any proportional action raises the loop gain close to the frequency where the phase approaches −180°, whereas integral action alone produces a low crossover; this consideration underlies the results of Section 6.

2.4 Study A: operating scenarios

Seven operating scenarios were defined (Table 4): the nominal point; input voltage ±20 %; load reduced to 50 % and 20 % of rated power; and inductance and capacitance reduced by 20 %. The duty ratio in each scenario was taken as $D = 1 - V_{\mathrm{in}}/V_{\mathrm{ref}}$. The scenarios cover the line, load and component-tolerance variations that typically degrade a nominal-point design. At 20 % load, Q rises to 5 and the resonance is sharply underdamped, whereas at V_{in} − 20 % the RHP-zero frequency falls to 20.4 Hz. Three scenarios (nominal, V_{in} − 20 % and 20 % load) form the optimization set, and the other four are held out and used only for validation.

2.5 Study B: laboratory-scale converter

Study B concerns a boost converter that was built on a perfboard and controlled by an Arduino Uno (Section 5) and that was modelled in MATLAB/Simulink with the same inductor, capacitor, load and switching frequency. Table 5 lists the parameters of the simulation model and of the hardware. In the simulation the nominal input voltage is $V_{\mathrm{in}} = 10\,\mathrm{V}$ and the reference is $V_{\mathrm{ref}} = V_{\mathrm{in}}/(1 - D) = 18.18\,\mathrm{V}$ for the design duty ratio $D = 0.45$; the load is $R = 50\,\Omega$ and the inductor has an estimated series

Table 5: Parameters of the laboratory-scale converter (Study B) in the simulation model and on the bench.

Item	Simulation model	Hardware
Input voltage	10 V nominal (±20 %)	7.2 V (laboratory supply)
Output reference	18.18 V	8, 12 and 15 V steps (5–18 V allowed)
Switching frequency	62.5 kHz	62.5 kHz
Inductance L	22.5 µH (±20 %)	same as simulation
Inductor series resistance R_L	0.1 Ω (estimated)	–
Capacitance C	470 µF (±20 %)	same as simulation
Load R	50 Ω (±50 %)	50 Ω (five 10 Ω in series)
Switch	IGBT model, $R_{on} = 1$ mΩ, $V_f = 1$ V	MOSFET with driver module
Diode	$R_{on} = 1$ mΩ, $V_f = 0.8$ V; snubber 500 Ω/250 nF	SR506
Controller	Continuous parallel PI, output limited to 0–0.9	Discrete PI at 1 kHz, duty limited to 200/255
Anti-windup	Clamping	Clamped incremental form
Voltage sensing	Ideal	Divider 40 kΩ/10 kΩ, 10-bit ADC

Table 6: Conduction mode of the laboratory-scale converter ($K = 2Lf_s/R = 0.056$). $D_{CCM} = 1 - V_{in}/V_o$; $D_{DCM} = \sqrt{KM(M-1)}$ with $M = V_o/V_{in}$ (ideal components).

Operating point	V_{in} (V)	V_o (V)	D_{CCM}	$K_{crit} = (1-D)^2$	D_{DCM}
Simulation, nominal	10	18.18	0.45	0.30	0.29
Hardware, 8 V level	7.2	8	0.10	0.81	0.08
Hardware, 12 V level	7.2	12	0.40	0.36	0.25
Hardware, 15 V level	7.2	15	0.52	0.23	0.36

resistance $R_L = 0.1\,\Omega$. On the bench the converter was supplied with 7.2 V and operated at reference levels between 8 V and 15 V (Section 5.5); the load is five 10 Ω resistors in series. The simulation and the hardware therefore share the power-stage components but not the input voltage, which is relevant for the interpretation of the comparison (Section 10).

2.6 Study B: averaged model and conduction mode

The optimizers of Study B use the same CCM averaged model as Study A, extended with the inductor resistance. With $D' = 1 - D$ and the duty ratio $D = 1 - V_{in}/V_{ref}$ of each scenario, the control-to-output transfer function is

$$G_{vd}(s) = \frac{a\,(1 - b\,s)}{c\,s^2 + b_L\,s + d_c}, \quad a = \frac{V_{in}}{D'^2},\ \ b = \frac{L}{RD'^2},\ \ c = \frac{LC}{D'^2},\ \ b_L = \frac{L/R + R_L C}{D'^2},\ \ d_c = 1 + \frac{R_L}{RD'^2}. \tag{5}$$

At the nominal point it has a DC gain $a = 33.1$ V, a resonance at 854 Hz with $Q \approx 1.2$ and an RHP zero at $1/(2\pi b) = 107$ kHz. In contrast with Study A, where the zero (31.8 Hz) coincides with the resonance, the zero of the laboratory-scale converter lies far above the switching frequency of 62.5 kHz and above any usable loop bandwidth, so it does not limit the design.

A more important point is the conduction mode. The CCM model is valid only if the normalized inductor-time constant $K = 2Lf_s/R$ exceeds its critical value $K_{crit} = (1-D)^2$ [1, 11]. For the parameters of Table 5, $K = 0.056$, whereas K_{crit} is 0.30 at the simulated design point and between 0.23 and 0.81 at the three hardware reference levels (Table 6). The inductor current is therefore discontinuous in the simulation and on the bench. The duty ratio required in discontinuous conduction is lower than the CCM value used in the averaged model: for the simulated reference of 18.18 V it is $D = \sqrt{KM(M-1)} = 0.29$ for an ideal converter, against 0.45 assumed by the averaged model, and the switching simulation indeed settles at a mean duty ratio of about 0.34 once the losses are included. The averaged CCM model is thus an approximation for Study B; its role is to provide a fast cost evaluation, and the switching model and the hardware are used to check the result.

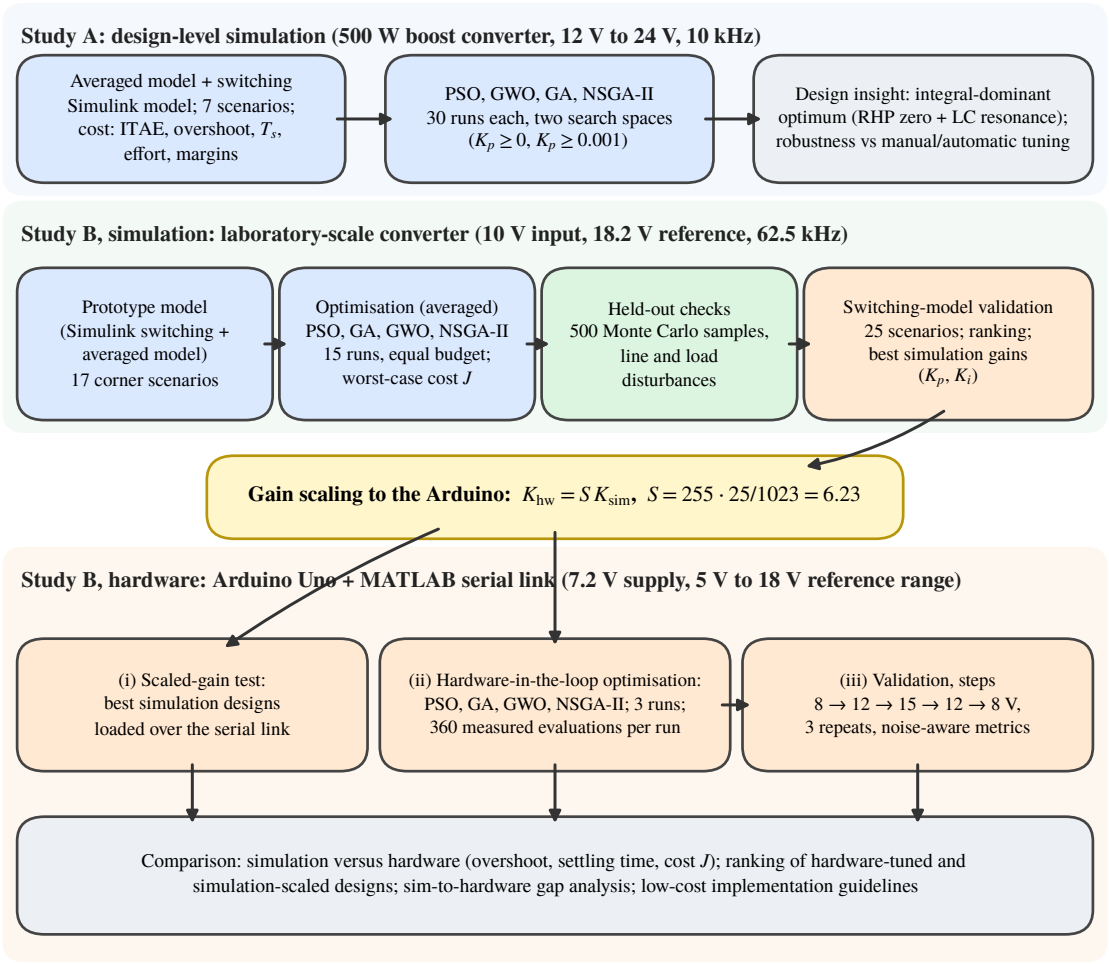


Figure 2: Overall framework of the study: design-level simulation (Study A), simulation of the laboratory-scale converter (Study B), gain scaling, and hardware experiments with an Arduino Uno controlled from MATLAB.

3 Overall Framework and Methodology of the Simulation Studies

3.1 Overall framework

The work follows the workflow of Fig. 2. Study A, a 500 W design-level converter, establishes the structure of a robust PI design and the relative merits of four metaheuristics. Study B then moves to a laboratory-scale converter and has three parts: (i) a simulation study that repeats the optimization with a cost and scenario set suited to the prototype and validates the result on the switching model; (ii) a gain-scaling step that converts the simulation gains into the integer arithmetic of the Arduino; and (iii) hardware experiments, in which the converter is tuned directly from MATLAB over the serial link and the scaled simulation gains are tested. The comparison between the simulated and the measured behaviour of the same gains is the central result of the paper.

All optimizers in both studies work on the same normalized decision vector: two numbers in $[0, 1]$ that are mapped linearly (for K_p) and logarithmically (for K_i, in Study B) to the gain bounds. Every cost function is a scalar built from the step-response metrics of Sections 3.3 and 4.3, and every stochastic comparison uses repeated independent runs and the Wilcoxon rank-sum test. The remainder of this section describes the methodology of Study A; Sections 4 and 5 describe Study B.

3.2 Study A: overview

The methodology of Study A has five stages: (i) definition of the converter, scenarios and baseline designs; (ii) formulation of the robust and multi-objective cost; (iii) optimization on the averaged model with PSO, GWO, GA and NSGA-II over repeated independent runs; (iv) re-ranking of the best candidates on the switching model; and (v) final evaluation on all scenarios, statistical analysis and selection. Two search spaces are considered: Case 1 ($K_\mathrm{p} \in [0, 0.03]$), in which the optimizer may choose a purely integral law, and Case 2 ($K_\mathrm{p} \in [0.001, 0.03]$), which enforces a genuine PI structure. In both cases $K_\mathrm{i} \in [0.1, 2]$.

3.3 Study A: performance metrics

For each scenario k, the output-voltage response v(t) to a step of the reference from 0 to V_ref over a horizon $T_\mathrm{s}\mathrm{top} = 0.4\,\mathrm{s}$ is evaluated through four normalized metrics. The normalized ITAE [52] is

$$\mathrm{ITAE_n} = \frac{\displaystyle\int_0^{T_\mathrm{stop}} t\ |V_\mathrm{ref} - v(t)|\ dt}{V_\mathrm{ref}\, T_\mathrm{stop}^2/2} \tag{6}$$

which equals 1 for a response that never leaves zero. The overshoot is

$$\mathrm{OS} = \frac{\max(0,\ \max_t v(t) - V_\mathrm{ref})}{V_\mathrm{ref}} \tag{7}$$

The settling time T_s is the last instant at which $|\mathrm{v} - V_\mathrm{ref}|$ exceeds 2 % of V_ref (set to $T_\mathrm{s}\mathrm{top}$ if the response never settles), reported as $T_\mathrm{s,n} = T_\mathrm{s}\ /\ T_\mathrm{s}\mathrm{top}$. The duty-cycle effort is the total variation of the duty ratio d sampled once per switching period $T_\mathrm{s}\mathrm{w} = 1/f_\mathrm{s}$, which removes PWM ripple while retaining the low-frequency trend:

$$E = \sum_j \big|\, d\big((j+1)T_\mathrm{sw}\big) - d\big(jT_\mathrm{sw}\big)\big| \tag{8}$$

Gain margin (GM) and phase margin (PM) are obtained from the loop gain of the averaged model. If the averaged closed loop is unstable, PM and GM are replaced by 0° and $-10\,\mathrm{dB}$, respectively, and a simulation that fails or diverges is assigned a fixed penalty.

3.4 Study A: cost functions

With the metric vector $\mathbf{m}_k = [\mathrm{ITAE_n},\ \mathrm{OS},\ T_\mathrm{s,n},\ E]$ of scenario k and weights $\mathbf{w} = [10,\ 5,\ 0.5,\ 0.2]$, the scenario cost is

$$J_k = \mathbf{w} \cdot \mathbf{m}_k + \omega_\mathrm{m} \left[\frac{\max(0,\ 45^\circ - \mathrm{PM}_k)}{45^\circ} + \frac{\max(0,\ 6\,\mathrm{dB} - \mathrm{GM}_k)}{6\,\mathrm{dB}} \right] \tag{9}$$

with $\omega_\mathrm{m} = 0.5$, and $J_k = 100$ for a failed simulation. The thresholds of 45° and 6 dB are common design targets [53]. The robust scalar cost combines the weighted mean and the worst case over the optimization scenarios $\mathcal{S}_\mathrm{opt}$:

$$J = (1-\lambda)\, \frac{\sum_{k\in\mathcal{S}_\mathrm{opt}} \alpha_k J_k}{\sum_{k\in\mathcal{S}_\mathrm{opt}} \alpha_k} + \lambda \max_{k\in\mathcal{S}_\mathrm{opt}} J_k \tag{10}$$

with $\lambda = 0.5$, $\alpha_\mathrm{nominal} = 2$ and $\alpha_k = 1$ for the other scenarios. For the multi-objective problem, the NSGA-II objective vector is

$$\mathbf{F} = \Big[\max_k \left(\mathrm{ITAE}_{\mathrm{n},k} + T_{\mathrm{s,n},k}\right) + \max_k \omega_\mathrm{m} \left(p_{\mathrm{PM},k} + p_{\mathrm{GM},k}\right),\ \max_k \mathrm{OS}_k,\ \max_k E_k \Big] \tag{11}$$

where $p_{\mathrm{PM},k}$ and $p_{\mathrm{GM},k}$ are the normalized margin penalties of (9). The weights were calibrated by evaluating the metrics for the baseline designs so that no single term dominated the cost.

Table 7: Cost-function and optimizer settings.

Item	Setting
Optimization scenarios $\mathcal{S}_{\text{opt}}$	Nominal, $V_{\text{in}} - 20\,\%$, Load $20\,\%$
Cost weights w (ITAE_{n}, OS, $T_{\text{s,n}}$, E)	10, 5, 0.5, 0.2
Margin penalty weight ω_{m}; targets	0.5; PM $\geq 45^{\circ}$, GM $\geq 6\,\text{dB}$
Mean/worst-case blend λ	0.5
Bounds (Case 1 / Case 2)	$K_{\text{p}} \in [0, 0.03]$ / [0.001, 0.03]; $K_{\text{i}} \in [0.1, 2]$
Population / iterations (PSO, GWO, GA)	12 / 20 (at most 240 cost evaluations per run)
PSO parameters	$c_1 = c_2 = 1.5$; w: $0.9 \to 0.4$; stall limit 8; seeded
GWO parameters	a: $2 \to 0$ linearly; unseeded
GA parameters	MATLAB defaults; seeded initial population
NSGA-II	population 40, 30 generations, MATLAB defaults
Independent runs	30 per optimizer and case (random seeds 1–30)

Table 8: Averaged (fast) versus switching (sim) metrics at the nominal scenario [ITAE_{n}, OS (pu), $T_{\text{s,n}}$, E].

Baseline	Averaged model	Switching model
Manual	0.0151, 0.0473, 0.416, 1.38	0.0226, 0.0053, 0.466, 0.618
Calculated	0.0329, 0, 0.510, 0.404	0.0538, 0.0044, 0.675, 0.494
PID Tuner	0.00425, 0.00125, 0.172, 0.500	0.0124, 0.0057, 0.245, 0.540

3.5 Study A: optimizers

All algorithms minimize (10) (or (11) for NSGA-II) over $(K_{\text{p}}, K_{\text{i}})$ with the bounds above. Table 7 lists the settings.

PSO [18] updates particle velocities and positions as

$$\mathbf{v}_i \leftarrow w\,\mathbf{v}_i + c_1 r_1\,(\mathbf{p}_i - \mathbf{x}_i) + c_2 r_2\,(\mathbf{g} - \mathbf{x}_i)\,, \qquad \mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i \tag{12}$$

with a linearly decreasing inertia weight w from 0.9 to 0.4 [54], $c_1 = c_2 = 1.5$, velocity clamping at ± 0.1 (ub − lb), position clamping to the bounds, and a stall criterion that stops the run after eight iterations without relative improvement above 10^{-6}. Three particles of the initial swarm are replaced by the three baseline designs (seeding), and the rest are uniformly random. **GWO** [30] uses wolves guided by the three best solutions with the control parameter decreasing linearly from 2 to 0, and starts from random positions. **GA** [31] is the MATLAB `ga` implementation with default selection, crossover and mutation operators and an initial population containing the baseline designs. **NSGA-II** [32] is the MATLAB `gamultiobj` implementation with a population of 40 and 30 generations; the knee solution is the Pareto point closest to the utopia point after normalizing the objectives.

3.6 Study A: two-stage evaluation on the averaged and switching models

Because a single switching simulation takes several seconds, thousands of evaluations per optimizer are impractical. Stage 1 therefore evaluates the cost (10) with the averaged model: the closed-loop step response of the averaged plant, scaled by V_{ref}, supplies the metrics of Section 3.3, and the loop gain supplies the margins. Stage 2 re-ranks the three best solutions of each optimizer on the switching model, using the same cost on the optimization scenarios, and retains the best; the retained designs and all baselines are then evaluated on all seven scenarios of the switching model. The agreement of the two models was checked at the nominal point for the three baseline designs (Table 8): both models give the same ranking of the controllers in ITAE_{n} and $T_{\text{s,n}}$, but the averaged model overestimates the overshoot, so the overshoot weight was kept moderate and all reported time-domain values come from the switching model.

3.7 Study A: baseline designs

Three reference designs are used to benchmark the optimized controllers. The **Manual** design ($K_\mathrm{p} = 0.0132$, $K_\mathrm{i} = 0.6867$) was obtained interactively with the MATLAB PID Tuner app [55] applied to the nominal averaged plant: the response-time and transient-behaviour settings were adjusted while the closed-loop step response was inspected visually, and the gains giving an acceptable response were retained.

The **Calculated** design places the crossover frequency at one tenth of the nominal RHP-zero frequency, $\omega_c = 2\pi f_z/10$, with the PI zero above the crossover at $\omega_z = 5\,\omega_c$, so that integral action dominates. With $C_\mathrm{n}(s) = (1 + s/\omega_z)/s$,

$$K_\mathrm{i} = \frac{1}{|C_\mathrm{n}(j\omega_c)\,G_{vd}(j\omega_c)|}, \qquad K_\mathrm{p} = \frac{K_\mathrm{i}}{\omega_z}, \tag{13}$$

which gives $K_\mathrm{p} = 0.0040$ and $K_\mathrm{i} = 0.4045$.

The **PID Tuner** design was obtained with the automatic tuning command of the MATLAB Control System Toolbox [55] (`pidtune(G,'PI')`) applied to the nominal averaged plant $G_{vd}(s)$ with a parallel-form PI structure and default settings, for which the crossover frequency is selected automatically and the target phase margin is 60°. It returned $K_\mathrm{p} = 0$ and $K_\mathrm{i} = 0.8837$, i.e. a pure integral controller. In Case 1 the optimizers search over $K_\mathrm{p} \in [0,\, 0.03]$ and $K_\mathrm{i} \in [0.1,\, 2]$, so the PID Tuner gains lie inside the search space. In Case 2 the lower bound is raised to $K_\mathrm{p} \geq 0.001$, so that every candidate retains a proportional term and the controller remains a PI controller; this places the PID Tuner baseline just outside the search space. The **Manual** design is the interactive result of the PID Tuner app, whereas the **PID Tuner** design is the non-interactive result of `pidtune` with default settings.

3.8 Statistical analysis

For each optimizer and case, the best, mean, standard deviation and worst final cost over the 30 runs are reported, together with the success rate, defined as the fraction of runs ending within 0.1 % of the best cost found by any optimizer, and the mean number of iterations needed to come within 0.1 % of the run's final cost. Differences between optimizers were tested with the two-sided Wilcoxon rank-sum test [56, 57] at a Bonferroni-corrected significance level of $0.05/3 \approx 0.0167$.

All simulations were run in MATLAB R2024a (version 24.1) on Windows 10 with the Control System, Global Optimization and Statistics and Machine Learning toolboxes, together with Simulink and Simscape Electrical.

3.9 Study A: computational effort

The study comprises 180 independent optimization runs (30 runs of each of the three single-objective optimizers in each of the two cases) and two NSGA-II runs. Every run starts from a different random seed (seeds 1–30, identical in the two cases), so the runs are independent apart from the shared seeding of the baseline designs in PSO and GA. Table 9 counts the cost evaluations and the simulations. Each cost evaluation covers the three optimization scenarios, that is, three closed-loop evaluations of the averaged model, so the optimization stage comprised at most about 4.5×10^4 cost evaluations (about 1.4×10^5 averaged-model scenario evaluations), PSO possibly stopping before its 240 evaluations. Only 166 switching-model simulations were needed: 27 for re-ranking, 49 for the final evaluation and 7 for the start-up plots in each case. At the measured 7.9 s per switching-model simulation, the switching stage required about 22 min, whereas running the whole optimization on the switching model would have required about 300 h (12 days), which motivates the two-stage procedure.

Table 9: Computational effort. Upper bounds computed from the optimizer settings; GA evaluations estimated from the default elite count.

Item	Per case	Both cases
PSO (30 runs, at most 240 evaluations each)	$\leq$ 7,200	$\leq$ 14,400
GWO (30 runs $\times$ 240 evaluations)	7,200	14,400
GA (30 runs $\times$ about 230 evaluations)	$\approx$ 6,900	$\approx$ 13,800
NSGA-II (one run, about 1,240 evaluations)	$\approx$ 1,240	$\approx$ 2,480
Total cost evaluations (averaged model)	$\approx$ 22,500	$\approx$ 45,100
Averaged-model scenario evaluations ($\times$ 3 scenarios)	$\approx$ 67,600	$\approx$ 135,200
Switching model: re-ranking (3 optimizers $\times$ 3 candidates $\times$ 3 scenarios)	27	54
Switching model: final evaluation (7 controllers $\times$ 7 scenarios)	49	98
Switching model: start-up plots (7 controllers)	7	14
Total switching-model simulations	83	166

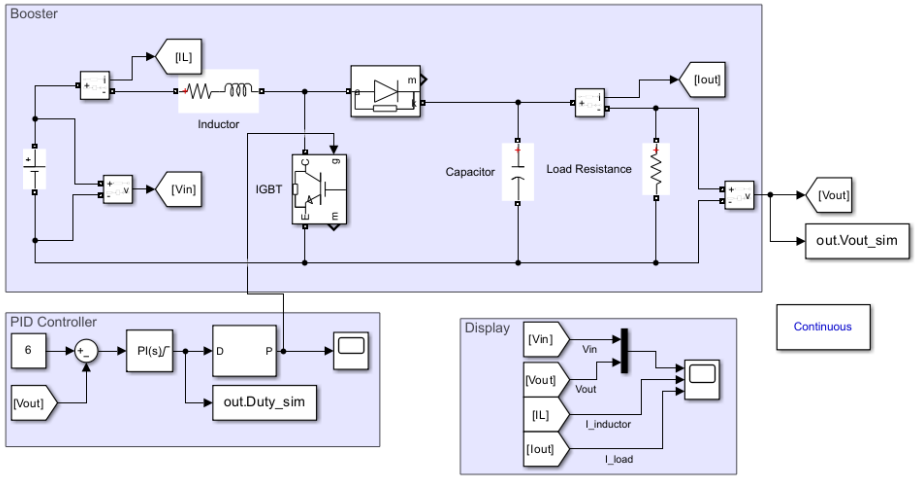


Figure 3: Simulink model of the laboratory-scale boost converter with the PI controller: power stage (source, inductor, switch, diode, capacitor, load), PI controller with PWM generator, and logging and display blocks.

4 Study B: Simulation Setup of the Laboratory-Scale Converter

4.1 Switching-level Simulink model

The converter of Table 5 was built in MATLAB/Simulink with Simscape Electrical blocks (Fig. 3): a DC voltage source, an inductor with series resistance R_L, an IGBT switch with on-state resistance 1 mΩ and forward voltage 1 V, a diode with on-state resistance 1 mΩ, forward voltage 0.8 V and a snubber of 500 Ω in series with 250 nF, the output capacitor (initially charged to the input voltage), and the load resistor. A DC–DC PWM generator at 62.5 kHz drives the switch. The controller is a continuous-time parallel-form PI block $C(s) = K_p + K_i/s$ acting on the difference between the reference and the measured output voltage; its output is the duty ratio, limited to $[0, 0.9]$ with clamping anti-windup. The model is solved by a variable-step solver with a relative tolerance of 10^{-3}. Output voltage and duty ratio are logged on a 20 μs grid, smoothed with a five-point moving average, and used to compute the metrics. Every parameter (V_{in}, L, C, R, K_p, K_i) is a workspace variable, so that the same model is run for every scenario and every design through `Simulink.SimulationInput`. A single simulation of 0.12 s takes about 8.6 s, against 0.135 s for one evaluation of the averaged-model cost.

4.2 Scenario set

Robustness is imposed through multiplicative uncertainty on four parameters: input voltage $V_{\text{in}} \in [0.8, 1.2]$, inductance $L \in [0.8, 1.2]$, capacitance $C \in [0.8, 1.2]$ and load resistance $R \in [0.5, 1.5]$ times their nominal values. The *corner* scenario set consists of the nominal point and the $2^4 = 16$ corners of this box (17 scenarios) and is used inside the optimization cost. A separate *Monte Carlo* set of 500 scenarios, sampled uniformly in the same box, is held out and used only for evaluation. For each scenario the duty ratio around which the averaged model is linearized is $D = 1 - V_{\text{in}}/V_{\text{ref}}$ with $V_{\text{ref}} = 18.18\,\text{V}$.

4.3 Metrics and cost function

For the averaged model the closed-loop step response of the normalized output (reference 1) is computed on a 500-point grid over 0.1 s; for the switching model the same normalization is applied to the logged output, over 0.12 s. Four metrics are extracted: the integral of the absolute error (IAE, in seconds), the overshoot OS (%), the 2 % settling time T_s and the 10–90 % rise time T_r. The phase margin PM is obtained from the loop gain of the averaged model, and a design whose closed loop is unstable in any scenario receives the fixed penalty $J = 10^3$. The scenario cost is

$$J_s = 0.3\,\frac{\text{IAE}_s}{0.01\,\text{s}} + 0.3\,\frac{T_{s,s}}{0.03\,\text{s}} + 0.4\,\frac{\text{OS}_s}{5\,\%}, \tag{14}$$

and the robust scalar cost combines the worst case, the mean and a phase-margin penalty,

$$J = \max_s J_s + 0.2\ \text{mean}_s\, J_s + 10\,\frac{\max(0,\ 45^\circ - \min_s \text{PM}_s)}{45^\circ}. \tag{15}$$

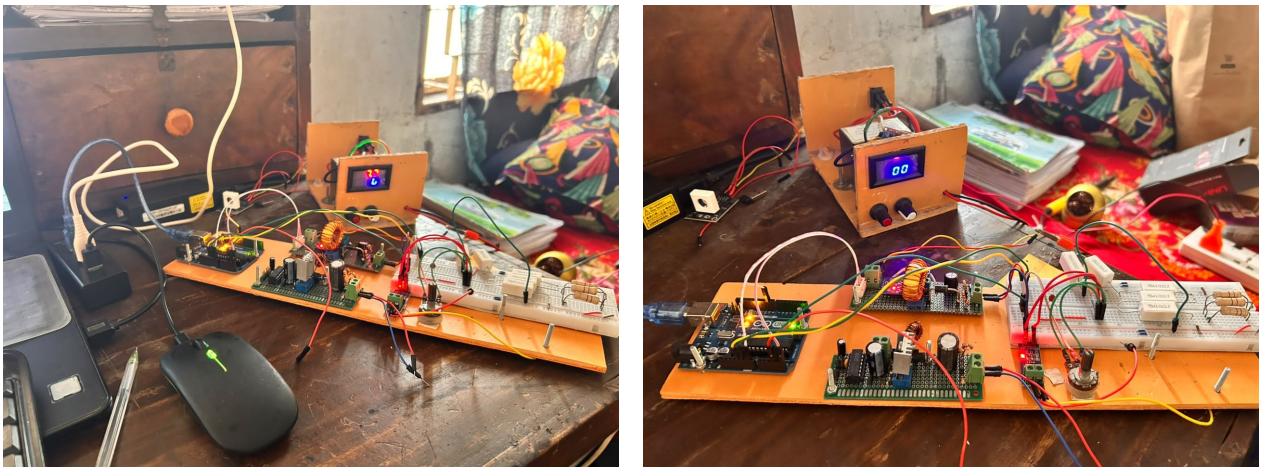
The normalizing constants (0.01 s, 0.03 s, 5 %) are of the order of the values obtained by the reference designs, so that the three terms contribute comparably. For NSGA-II the objective vector is $[\max_s \text{IAE}_s/0.01,\ \max_s T_{s,s}/0.03,\ \max_s \text{OS}_s/5]$ plus the margin penalty, and the knee of the front is the point with the largest distance from the hyperplane through the anchor points of the normalized front.

4.4 Optimizers and computational budget

The decision vector is (K_p, K_i) with $K_p \in [0, 0.03]$ and $K_i \in [1.5, 150]$ (logarithmic scale). The four optimizers were implemented in MATLAB with the same normalized coding. PSO uses a linearly decreasing inertia weight from 0.9 to 0.4, acceleration coefficients $c_1 = c_2 = 1.6$ and a velocity limit of 0.25 of the normalized range; the GA uses binary tournament selection, simulated binary crossover (probability 0.9, distribution index 15), polynomial mutation (probability 0.3, index 20) and two elite individuals; GWO follows the standard algorithm with the control parameter decreasing linearly from 2 to 0; and NSGA-II uses non-dominated sorting with crowding distance and the same crossover and mutation operators as the GA. No optimizer was seeded with a reference design. To compare the algorithms fairly, all four received the same budget: a population of 17 and 39 iterations, that is 680 cost evaluations per run, for 15 independent runs each, with the random seed set from the algorithm and run index. In total the optimization stage comprised $4 \times 15 \times 680 = 40{,}800$ cost evaluations (about 74 min). The population, the number of iterations and the number of runs were chosen automatically by the script from the measured evaluation time and a total time budget of 170 min.

4.5 Reference designs

Five reference designs were included. Three are `pidtune` PI designs for the nominal averaged plant with target crossover frequencies of 100, 150 and 300 rad/s ($K_p = 0$, $K_i = 3.04$, 4.57 and 9.12). Two come from the PID Tuner app of MATLAB: its automatic design ($K_p = 0$, $K_i = 48.06$) and a design obtained by moving the response-time slider ($K_p = 0.00243$, $K_i = 15.31$). They represent the usual practice of a designer who relies on the standard tools.



(a) **(b)**

Figure 4: Photographs of the experimental set-up: the Arduino Uno, the boost power stage with its driver, the resistive load on the breadboard, the potentiometer for the local reference and the laboratory supply with its display.

4.6 Validation, Monte Carlo and disturbance tests

The best design of each optimizer (the run with the lowest J) and the five references were evaluated in four ways. (i) *Corner evaluation* on the averaged model, giving the worst-case metrics. (ii) *Monte Carlo evaluation* on the 500 held-out scenarios with the acceptance specification OS $\leq 10\,\%$, $T_s \leq 20\,\text{ms}$ and PM $\geq 45°$; the percentage of scenarios meeting it is the success rate. (iii) *Disturbance rejection* on the averaged model: a $-20\,\%$ step of the input voltage and a $+20\,\%$ step of the load current, with the peak deviation expressed in percent of V_{ref} and the time to return within $2\,\%$. (iv) *Switching-model validation*: each design was simulated in Simulink in 25 scenarios (the nominal point, the 15 corners with the highest mean cost over all designs, and 9 random held-out scenarios), giving $9 \times 25 = 225$ simulations. The designs were ranked by the worst-case cost J of (iv). The statistical analysis follows Section 3.8, with the Bonferroni level $0.05/6$ for the six pairwise comparisons of four optimizers.

5 Study B: Hardware Platform, Gain Scaling and Experimental Methodology

5.1 Experimental platform

Fig. 4 shows the experimental set-up and Fig. 5 its block diagram. The power stage is a boost converter built on a perfboard, comprising a toroidal inductor, a MOSFET driven through a driver module, an SR506 Schottky rectifier, the output capacitor and a resistive load of $50\,\Omega$ (five $10\,\Omega$ resistors in series); the inductor and capacitor are those of the simulation model (Table 5). A laboratory supply set to 7.2 V feeds the converter. An Arduino Uno (ATmega328P) [58] generates the PWM signal, measures the output voltage through a resistive divider ($40\,\text{k}\Omega$ above $10\,\text{k}\Omega$, so that the ADC input is one fifth of the output voltage) on analog pin A5, and runs the PI controller. A potentiometer on pin A4 sets the reference locally, between 5 V and 18 V, when no computer is connected. The Arduino is connected by USB to a laptop that runs MATLAB. No dSPACE, OPAL-RT, STM32 or other real-time control hardware is used: the real-time task (sampling, control law and PWM) runs on the 8-bit microcontroller, and the laptop only supervises the experiment.

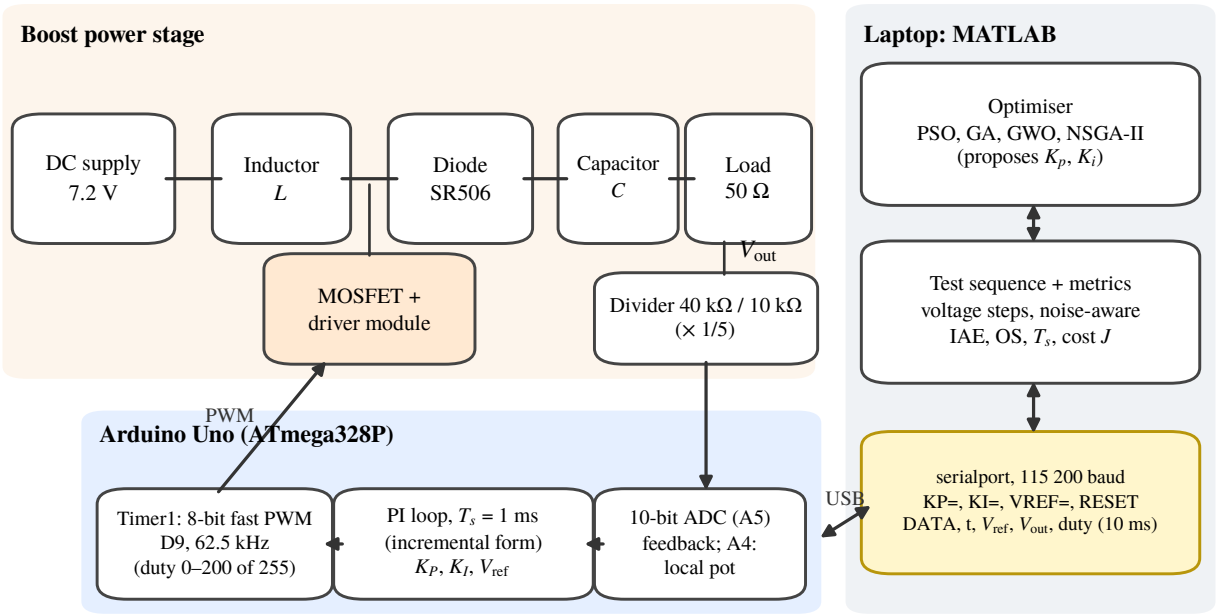


Figure 5: Block diagram of the hardware: boost power stage, Arduino Uno with Timer1 PWM, PI loop and ADC, and the MATLAB supervisory computer connected by the serial link.

5.2 Firmware and digital PI controller

Timer1 of the ATmega328P is configured in 8-bit fast PWM mode with no prescaler, which gives a PWM frequency of 16 MHz/256 = 62.5 kHz on pin D9 (OC1A) and a duty-ratio resolution of 1/255. The duty command is limited to 200/255 = 0.784. The control loop is executed every $T_s = 1$ ms (paced with `micros()`); in each cycle the ADC is read, the error in ADC counts $e_k = e_{\text{ref}} - e_{\text{meas}}$ is formed, and the PI law is evaluated in incremental (velocity) form,

$$u_k = u_{k-1} + K_P\,(e_k - e_{k-1}) + K_I\,T_s\,e_k, \qquad 0 \le u_k \le 200, \tag{16}$$

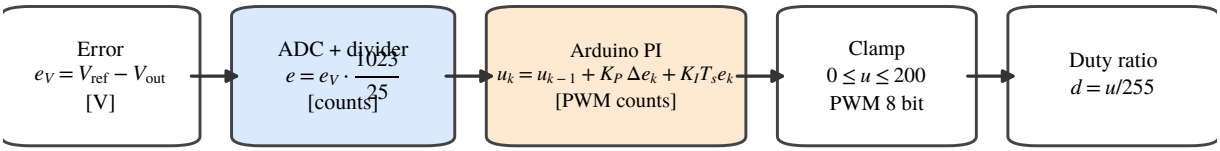
where u_k is the PWM compare value (duty counts). The reference in ADC counts is $e_{\text{ref}} = V_{\text{ref}}/(5 \cdot 5) \cdot 1023$. Because u_{k-1} is stored after saturation, the incremental form provides anti-windup without additional logic. The variable names K_P and K_I (capitals) denote the gains in Arduino units, as distinct from the simulation gains K_p and K_i.

5.3 Serial protocol and MATLAB supervision

MATLAB opens the port with `serialport` at 115 200 baud and keeps it open during the whole experiment. Commands are text lines: `KP=`, `KI=` and `VREF=` set the gains and the reference (the sketch accepts $0 \le K_P \le 5$, $0 \le K_I \le 500$ and $5 \le V_{\text{ref}} \le 18$ V and answers with an acknowledgement), and `RESET` clears the controller memory and the duty. The Arduino streams a telemetry line every 10 ms containing the time stamp, the reference, the measured output voltage, the duty in percent and the PWM compare value. A test runs entirely on the Arduino once its parameters are loaded, so the serial link carries only short commands and telemetry and does not affect the 1 kHz control loop.

5.4 Scaling of the simulation gains to Arduino units

The simulation PI law acts on the error in volts and produces a duty ratio, $d = K_p\,e_V + K_i \int e_V\,\mathrm{d}t$, while the firmware acts on the error in ADC counts and produces PWM counts (Fig. 6). With the divider ratio $(R_{\text{top}} + R_{\text{bot}})/R_{\text{bot}} = 5$ and a 5 V ADC reference, one ADC count corresponds to 25/1023 V at the converter output, so $e = e_V \cdot 1023/25$. The duty ratio is related to the PWM



Simulation PI: $d = K_\mathrm{p}\, e_V + K_\mathrm{i} \int e_V\, dt \quad \Rightarrow \quad K_P = K_\mathrm{p} \cdot 255 \cdot \dfrac{25}{1023}, \quad K_I = K_\mathrm{i} \cdot 255 \cdot \dfrac{25}{1023}$ (same factor $S = 6.23$ for both gains)

Figure 6: Signal chain of the hardware controller and the origin of the scaling factor S that converts the simulation gains into Arduino gains.

counts by $u = 255\, d$. Equating the two laws gives

$$K_P = S\, K_p, \qquad K_I = S\, K_i, \qquad S = 255 \cdot \frac{25}{1023} = 6.23. \tag{17}$$

The same factor applies to both gains because the integral term of (16) already contains the sampling period T_s. For the best simulation design $K_p = 0.01614$, $K_i = 44.96$ this gives $K_P = 0.1006$ and $K_I = 280.2$, which lies inside the limits accepted by the firmware. Equation (17) converts the units only; it does not compensate for the difference between the simulation model and the real converter, which is the subject of Section 9.

5.5 Hardware test and metrics

One hardware *evaluation* consists of loading the gains, setting the first reference level and waiting 0.5 s, recording a 0.25 s pre-step window, and then applying the sequence of reference steps while recording 0.8 s after each step. During the optimization the levels are 8 V → 12 V → 15 V (two up-steps; 2.4 s per evaluation including the communication); in the validation the sequence is 8 → 12 → 15 → 12 → 8 V (four steps), so that down-steps, which depend on the discharge of the output capacitor through the load, are also tested. If the output exceeds 17.5 V the test is aborted and the controller returns to a safe, known-good gain pair ($K_P = 0.1$, $K_I = 20$).

For each step the measured output is smoothed with a three-point moving average and normalized to $y = (v - y_0)/(V_\mathrm{to} - y_0)$, where y_0 is the mean of the pre-step window. The following metrics are extracted: IAE (integral of $|1 - y|$), overshoot relative to the final value, 10–90 % rise time T_r (by linear interpolation), steady-state error, output ripple (standard deviation of y over the last 0.3 s, in percent of the step), and the standard deviation of the duty. The settling time uses a *noise-aware* band $\max\{2\,\%,\, 2\sigma_N/|\Delta V|\}$, where σ_N is the white-noise level estimated from the differences of the final 0.3 s and ΔV is the step height, because a fixed 2 % band is comparable to the measurement noise of a 10-bit ADC at low step heights. The cost mirrors that of the simulation,

$$J_k = 0.3\,\frac{\mathrm{IAE}_k}{0.05\,\mathrm{s}} + 0.3\,\frac{T_{s,k}}{0.2\,\mathrm{s}} + 0.4\,\frac{\mathrm{OS}_k}{5\,\%} + p_k, \qquad J = \max_k J_k + 0.2\ \mathrm{mean}_k\, J_k, \tag{18}$$

with the penalty $p_k = \max(0, e_{\mathrm{ss},k} - 2) + \max(0, \mathrm{ripple}_k - 5)$ (both in percent) and a fixed cost of 10^3 for an invalid or aborted test.

5.6 Hardware-in-the-loop optimization

The four optimizers of Section 4.4 were run in MATLAB with the measured cost (18) as the objective: every cost evaluation is one hardware evaluation of 2.4 s. The search space is $K_P \in [0, 0.5]$ (linear) and $K_I \in [2, 200]$ (logarithmic), in Arduino units, which contains the gains obtained by scaling the

simulation designs except those with $K_I > 200$. The population was 6 and the number of iterations 59, so that each run needs 360 measured evaluations (14.3 min); three independent runs were made for each optimizer, in sequence, with a rest of 4 s between runs to protect the load resistors. The whole experiment took 175 min. The best design of each run was then re-evaluated twice and the lowest re-evaluated cost was taken as its result, which reduces the selection bias of the minimum of a noisy cost.

5.7 Validation and statistics

The best design of each optimizer and two reference gain pairs measured on the bench ($K_P = 0.1$, $K_I = 20$, the pair used for safe recovery, and $K_P = 0.05$, $K_I = 10$) were validated with the four-step sequence, three repeats per design. The reported metrics are the worst over the four steps, averaged over the repeats, with the standard deviation over the repeats; the ensemble-averaged traces are also analysed, which lowers the noise by $\sqrt{3}$. Because only three runs per optimizer are possible within the time available, the rank-sum p-values are reported for completeness only and no significance is claimed for the hardware comparison.

5.8 Bench re-test of the scaled simulation designs

Four designs from the simulation study were scaled with (17) and loaded into the Arduino through the serial link: the best simulation design (GWO), the NSGA-II knee, the `pidtune` design with $\omega_c = 300$ rad/s, and the slider design of the PID Tuner app. Two derated versions of the best simulation design (integral gain divided by 3 and by 6, proportional gain unchanged) and two controls (the hardware GA design and the baseline gains) were added. Each of the eight designs was measured five times with the four-step validation sequence of Section 5.5, in random order within each repetition so that a slow drift of the supply or of the load resistors does not favour one design, using the same measurement and cost functions as the optimization; the results are the worst case over the four steps, averaged over the repeats, with the standard deviation of J and of the overshoot. Because the repeat-to-repeat spread of J is large, Welch and Mann–Whitney tests are given for selected pairs and no multiple-comparison correction is applied. The measured overshoot, rise time, steady-state error and cost are compared with the corresponding simulation in Section 9.

5.9 Simulation with a sampled controller

To test whether the difference between the continuous simulated controller and the sampled firmware explains the transfer gap, the Simulink model of Section 4.1 was run with the PI block switched to discrete time ($T_s = 1$ ms, forward-Euler integration, clamping anti-windup retained), in two configurations. In Part A the start-up of the nominal scenario ($V_{\text{in}} = 10$ V, $V_{\text{ref}} = 18.18$ V, 0.12 s) was simulated with the continuous and with the sampled controller, and the overshoot and settling time were extracted as in Section 4.3. In Part B the input voltage was set to 7.2 V and the constant reference was replaced by a staircase 8→12→15→12→8 V of 0.25 s per level; overshoot, rise time, settling time, steady-state error and the cost (18) (without the noise penalties, since the simulation is noise-free) were computed per step with the same definitions as for the bench, with the pre-step level taken from the last 20 % of the previous level. The model copy used for these runs was a copy of the original, so that the simulation of Section 4 was not altered. Eight designs were simulated: the four scaled designs, the two derated designs, the hardware GA design and the baseline gains (gains in simulation units obtained by dividing the Arduino gains by S).

6 Results of Study A: 500 W Design-Level Simulation

6.1 Case 1: robust tuning without a lower bound on K_{p}

Worst-case performance. Table 10 summarizes the worst case over the seven scenarios. The designs tuned manually or by the nominal-point automatic tuner were not robust. The Manual design

Table 10: Worst-case performance over seven scenarios, Case 1 ($K_p \geq 0$). NS = not settled within 0.4 s; "unstable" = at least one scenario unstable in the averaged model.

Controller	K_p	K_i	$\mathrm{ITAE}_{n,max}$	**Overshoot$_{max}$ (%)**	$T_{s,max}$ **(ms)**	PM_{min} (°)	GM_{min} **(dB)**
Manual	0.0132	0.6867	0.0887	18.68	NS (>400)	unstable	unstable
Calculated	0.0040	0.4045	0.0598	0.633	288.8	33.7	4.98
PID Tuner	0	0.8837	0.0766	15.21	NS (>400)	unstable	unstable
PSO	**0**	**0.4086**	**0.0553**	**0.758**	**254.3**	**67.5**	**5.85**
GWO	0	0.4090	0.0551	0.760	254.0	67.5	5.84
GA	0	0.4087	0.0552	0.758	254.2	67.5	5.85
NSGA-II (knee)	0.0036	0.3047	0.1038	0.350	373.6	85.5	7.11

Table 11: Overshoot (%) / settling time (ms) per scenario, Case 1. NS = not settled within 0.4 s.

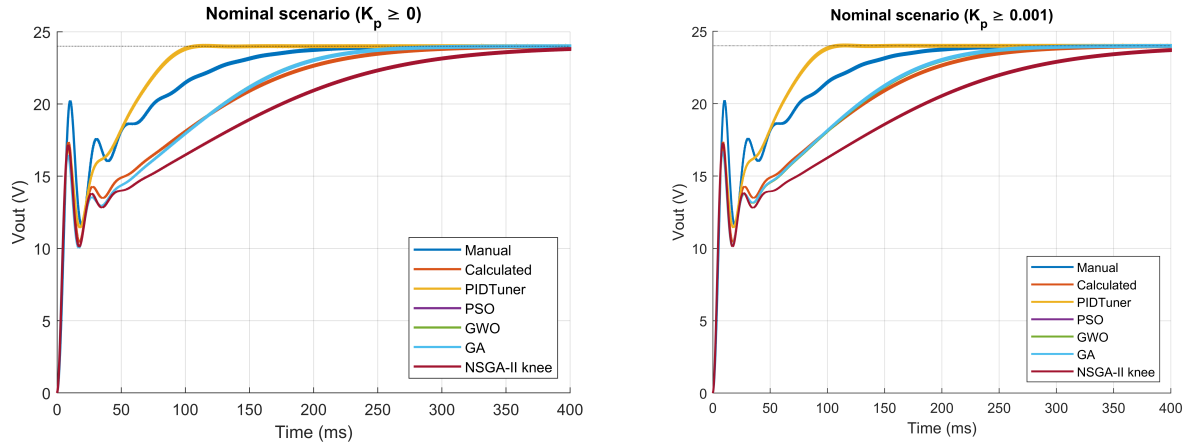
Scenario	**Manual**	**Calculated**	**PID Tuner**	**PSO**	**NSGA-II (knee)**
Nominal	0.53 / 186.4	0.44 / 269.9	0.63 / 95.7	0.52 / 239.2	0.00 / 361.2
V_{in} − 20 %	1.16 / 169.3	0.63 / 241.8	13.24 / 391.3	0.76 / 203.9	0.35 / 328.7
V_{in} + 20 %	0.63 / 189.5	0.20 / 280.5	0.44 / 109.1	0.35 / 254.3	0.00 / 373.6
Load 50 %	0.98 / 221.5	0.11 / 271.7	0.38 / 113.2	0.21 / 244.2	0.00 / 358.5
Load 20 %	18.68 / NS	0.08 / 288.8	15.21 / NS	0.10 / 248.5	0.00 / 363.9
L − 20 %	0.53 / 189.9	0.42 / 274.0	0.54 / 101.4	0.51 / 244.1	0.00 / 365.3
C − 20 %	0.67 / 190.9	0.57 / 274.4	0.81 / 96.7	0.66 / 242.6	0.00 / 367.4

reached 18.7 % overshoot at 20 % load and did not settle within the 0.4 s horizon in that scenario. The PID Tuner design, although the fastest at the nominal point, produced 13.2 % overshoot at V_{in} − 20 % and 15.2 % overshoot at 20 % load, and also failed to settle at 20 % load. In the averaged model both designs had at least one scenario with an unstable closed loop. The Calculated design was stable in all scenarios, with a worst-case overshoot of 0.63 %, a worst-case settling time of 288.8 ms and a minimum phase margin of 33.7°.

The PSO, GWO and GA designs converged to practically the same controller: $K_p = 0$ and K_i between 0.4086 and 0.4090, an integral-only law. Relative to the Calculated design, the PSO design reduced the worst-case ITAE_n from 0.0598 to 0.0553 (−7.6 %) and the worst-case settling time from 288.8 ms to 254.3 ms (−12.0 %), and raised the minimum phase margin from 33.7° to 67.5° and the minimum gain margin from 4.98 dB to 5.85 dB. The price was a small increase in the worst-case overshoot, from 0.63 % to 0.76 %. Compared with the Manual and PID Tuner designs, the worst-case overshoot fell by factors of about 25 and 20, respectively. The PSO design satisfied the 45° phase-margin target with a wide margin, while its gain margin (5.85 dB) remained marginally below the 6 dB soft target, a consequence of the trade-off with settling speed discussed in Section 10.

Scenario-wise behaviour. Table 11 gives the overshoot and settling time in each scenario. The PSO design settled 9.3–15.7 % faster than the Calculated design in every scenario (for example 239.2 ms against 269.9 ms at the nominal point and 248.5 ms against 288.8 ms at 20 % load), while its overshoot remained below 0.8 % everywhere. The Calculated design had slightly lower overshoot than PSO in all seven scenarios, by at most 0.15 percentage points. The spread of the PSO settling times across scenarios was narrow, from 203.9 ms to 254.3 ms. The optimizers used only three scenarios in their cost function (nominal, V_{in} − 20 % and 20 % load); in the four held-out scenarios (V_{in} + 20 %, 50 % load, L − 20 %, C − 20 %), the PSO design gave overshoots between 0.21 % and 0.66 % and settling times between 242.6 ms and 254.3 ms, so the gain in speed was not confined to the scenarios seen during optimization. The GWO and GA designs were indistinguishable from the PSO design at the resolution of the tables (overshoot differences below 0.02 percentage points and settling-time differences below 0.4 ms) and are therefore not tabulated separately.

Start-up response. Fig. 7(a) shows the nominal start-up response of all controllers. All designs share an initial transient in which the output rises to roughly 17–20 V within about 10 ms and then



(a) Case 1 **(b)** Case 2

Figure 7: Nominal start-up response of the output voltage for all controllers: (a) Case 1 and (b) Case 2. The PSO, GWO and GA responses overlap.

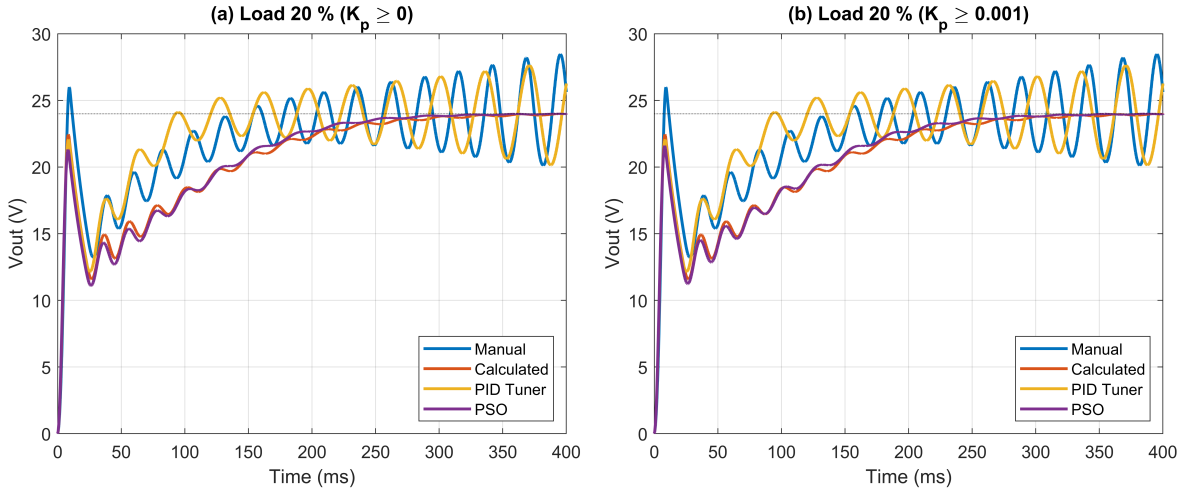


Figure 8: Start-up response at 20 % load: (a) Case 1 and (b) Case 2. The Manual and PID Tuner designs oscillate with growing amplitude and do not settle within 0.4 s.

dips to 10–12 V near 17 ms, which is consistent with the excitation of the converter's open-loop LC resonance during start-up; the controller gains therefore mainly shape the slower, integral-dominated approach to 24 V. The PID Tuner design reached the reference in about 100 ms at the nominal point, but this advantage disappears at light load (Fig. 8), the Manual design showed the largest initial peak (about 20 V) and a visible secondary oscillation, and the PSO, GWO and GA responses overlap and approach 24 V monotonically without visible overshoot at this scale. The NSGA-II knee design was the slowest and the most heavily damped.

Start-up at light load. At 20 % load (Fig. 8) the Manual and PID Tuner designs oscillate around the reference with growing amplitude that is still increasing at the end of the 0.4 s horizon, so neither settles and their overshoots (18.7 % and 15.2 %) are lower bounds. The Calculated and PSO designs show decaying oscillations and settle in 288.8 ms and 248.5 ms (Case 1). The PSO designs of the two cases are almost identical (248.5 ms and 247.5 ms), so the proportional gain of Case 2 has little effect here. The GWO and GA designs coincide with PSO and are not plotted.

6.2 Reliability and convergence of the optimizers

All three optimizers attained the same best cost in Case 1, $J = 0.6261$, so none found a better solution than the others; they differed in how reliably they reached it (Table 12 and Fig. 9). PSO

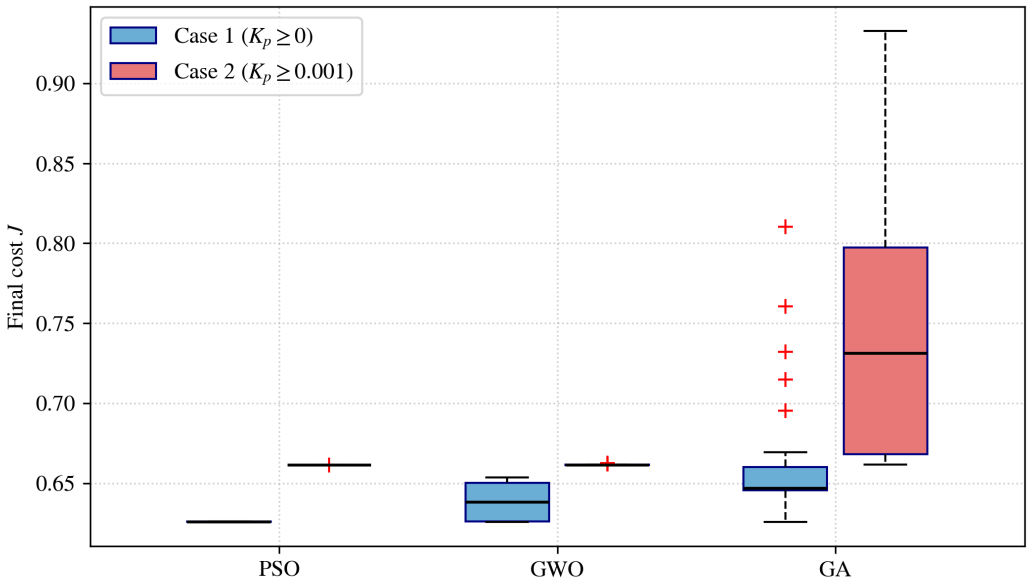


Figure 9: Distribution of the final cost J over 30 independent runs for PSO, GWO and GA in Case 1 and Case 2 (box: interquartile range; line: median; whiskers: 1.5 times the interquartile range; plus signs: outliers).

Table 12: Statistical comparison of the optimizers over 30 independent runs on the final cost J. Success = runs within 0.1 % of the best cost found by any optimizer. Mean iterations to converge = mean number of iterations needed to come within 0.1 % of the run's final cost (n/a: not reported for GA). The corresponding p-values are given in Table 13.

Case	Optimizer	Best	Mean	Std	Worst	Success	Mean iterations to converge
Case 1 ($K_\mathrm{p} \geq 0$)	PSO	0.6261	0.6261	$< 10^{-4}$	0.6262	30/30 (100 %)	8.6
	GWO	0.6261	0.6386	0.0120	0.6536	10/30 (33.3 %)	7.9
	GA	0.6261	0.6602	0.0426	0.8103	7/30 (23.3 %)	n/a
Case 2 ($K_\mathrm{p} \geq 0.001$)	PSO	0.6615	0.6615	$< 10^{-4}$	0.6616	30/30 (100 %)	7.5
	GWO	0.6615	0.6617	0.0003	0.6626	26/30 (86.7 %)	8.2
	GA	0.6617	0.7458	0.0821	0.9326	1/30 (3.3 %)	n/a

reached the best cost, to within 0.1 %, in all 30 runs (success rate 100 %), with a mean cost of 0.6261 and a standard deviation below 10^{-4}. GWO reached it in 10 of 30 runs (33.3 %), with a mean of 0.6386 and a standard deviation of 0.0120. GA reached it in 7 of 30 runs (23.3 %), with a mean of 0.6602, a standard deviation of 0.0426 and a worst-case cost of 0.8103. The two-sided Wilcoxon rank-sum test (Table 13), evaluated at the Bonferroni-corrected level of $0.05/3 \approx 0.0167$ (Section 3.8), showed that the final-cost distribution of PSO differed significantly from those of GWO ($p = 9.8 \times 10^{-10}$) and GA ($p = 1.6 \times 10^{-8}$), whereas GWO and GA did not differ significantly from each other ($p = 0.264$). The grouped boxplots of Fig. 9 show the same picture: the PSO boxes collapse to a line in both cases, the GWO boxes are narrow, and the GA boxes are wide with outliers. The corresponding results for Case 2 are discussed in Section 6.4.

The convergence curves (Fig. 10, mean best cost over 30 runs for both cases) show that PSO started at a low cost (about 0.9) because its initial swarm contained the baseline designs, whereas GWO started from random positions at a mean cost near 8 in Case 1 and near 10 in Case 2, presumably because several initial candidates were unstable and heavily penalized, and GA started at about 2.9 and 3.1. All three algorithms reached the same plateau. PSO and GWO came within 0.1 % of their final cost after a similar number of iterations (8.6 and 7.9 on average in Case 1, 7.5 and 8.2 in Case 2), and the GA curve lay above the other two until about iteration 8–10 before merging with them.

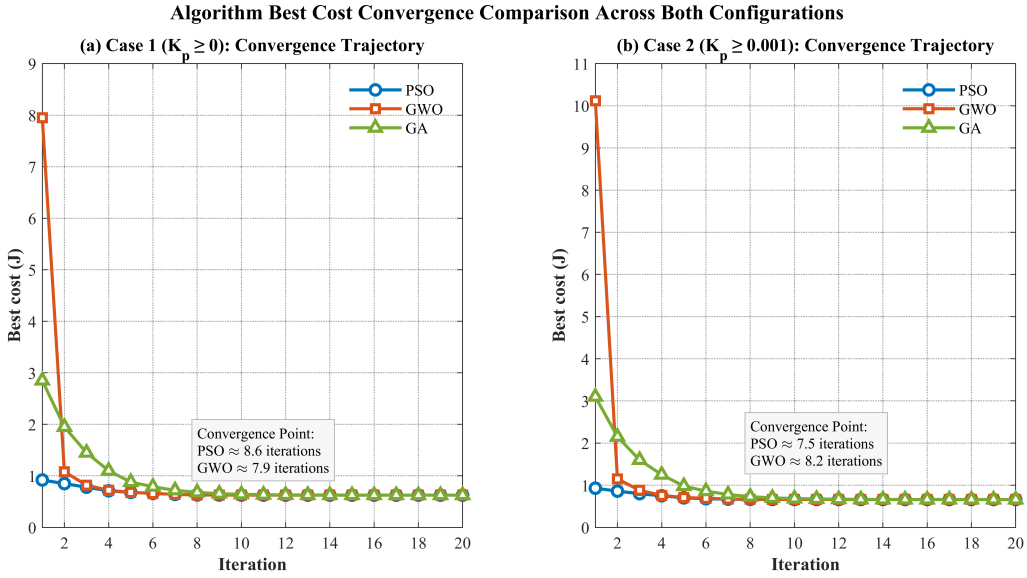


Figure 10: Mean best-cost convergence of PSO, GWO and GA over 30 independent runs: (a) Case 1 ($K_\mathrm{p} \geq 0$) and (b) Case 2 ($K_\mathrm{p} \geq 0.001$). The annotated values are the mean numbers of iterations needed by PSO and GWO to come within 0.1 % of their final cost.

Table 13: Two-sided Wilcoxon rank-sum p-values for the final cost J (30 runs per optimizer, computed with MATLAB `ranksum`). With the Bonferroni correction for three comparisons, a difference is significant when $p < 0.05/3 \approx 0.0167$; n.s. = not significant.

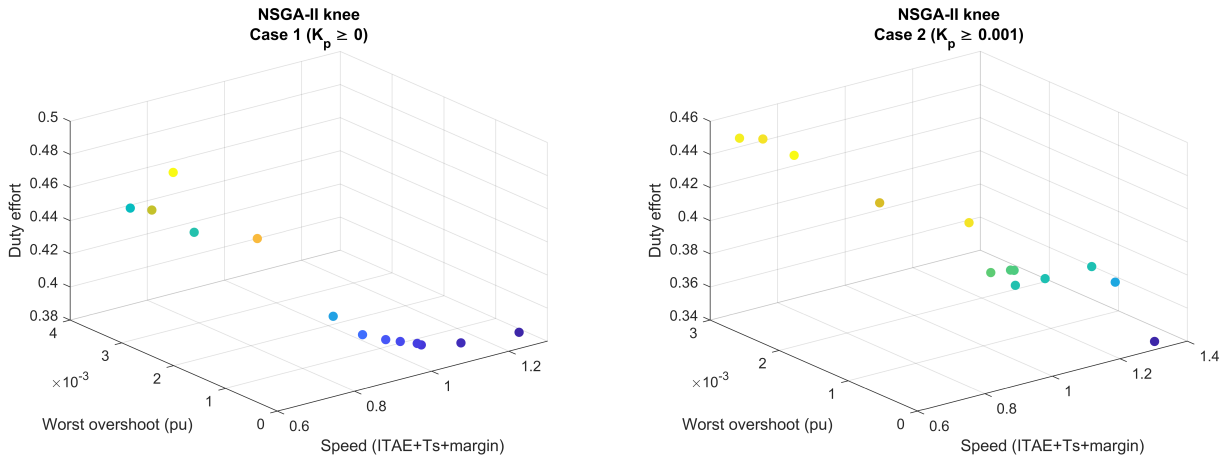
Comparison	**Case 1** ($K_\mathrm{p} \geq 0$)	**Case 2** ($K_\mathrm{p} \geq 0.001$)
PSO vs GWO	9.8×10^{-10}	8.3×10^{-6}
PSO vs GA	1.6×10^{-8}	3.0×10^{-11}
GWO vs GA	0.264 (n.s.)	6.7×10^{-11}

6.3 Multi-objective tuning (Case 1)

NSGA-II was used to explore the trade-off between speed ($\mathrm{ITAE_n} + T_\mathrm{s,n}$ + margin penalty), worst-case overshoot and duty-cycle effort. The resulting Pareto front (Fig. 11(a)) contained about a dozen non-dominated designs, all with worst-case overshoot below 0.4 %, a duty effort between roughly 0.38 and 0.47, and a speed index spanning approximately 0.6 to 1.2, so that no single design minimized all three objectives. The knee point, selected as the solution closest to the normalized utopia point, had $K_\mathrm{p} = 0.0036$ and $K_\mathrm{i} = 0.3047$. It reduced the worst-case overshoot to 0.35 % and raised the margins to $\mathrm{PM_{min}} = 85.5°$ and $\mathrm{GM_{min}} = 7.11\,\mathrm{dB}$, which satisfies both margin targets, but its worst-case settling time rose to 373.6 ms, 47 % longer than that of the PSO design. NSGA-II was run once, so the front is illustrative and no statistical statement is made about it.

6.4 Case 2: a genuine PI controller ($K_\mathrm{p} \geq 0.001$)

Because the unconstrained optimum in Case 1 degenerates to an integral controller, the search was repeated with $K_\mathrm{p} \geq 0.001$ so that the result remains a PI controller. The Manual, Calculated and PID Tuner reference designs are unchanged. All three optimizers placed K_p at or very near the bound (0.0010–0.0011) with K_i between 0.4136 and 0.4164 (Table 14). Relative to the Calculated design, the PSO design reduced the worst-case $\mathrm{ITAE_n}$ by 9.1 % (0.0598 to 0.0544) and the worst-case settling time by 11.3 % (288.8 ms to 256.1 ms), lowered the worst-case overshoot below that of the unconstrained design (0.652 % against 0.758 %), and gave a minimum phase margin of 70.7° and a minimum gain margin of 5.68 dB. Imposing the bound increased the optimized cost by 5.7 % (0.6261 to 0.6615), which quantifies the price of insisting on a non-zero proportional term. The per-scenario



(a) Case 1 **(b)** Case 2

Figure 11: NSGA-II Pareto fronts in the space of speed index ($\mathrm{ITAE_n} + T_{\mathrm{s,n}}$ + margin penalty), worst-case overshoot and duty-cycle effort: (a) Case 1 and (b) Case 2.

Table 14: Worst-case performance over seven scenarios, Case 2 ($K_\mathrm{p} \geq 0.001$). Reference designs are unchanged from Table 10. NS = not settled within 0.4 s; "unstable" = at least one scenario unstable in the averaged model.

Controller	K_p	K_i	$\mathrm{ITAE_{n,max}}$	**Overshoot$_{\max}$ (%)**	$T_{\mathrm{s,max}}$ **(ms)**	$\mathrm{PM_{min}}$ (°)	$\mathrm{GM_{min}}$ **(dB)**
Manual	0.0132	0.6867	0.0887	18.68	NS (>400)	unstable	unstable
Calculated	0.0040	0.4045	0.0598	0.633	288.8	33.7	4.98
PID Tuner	0	0.8837	0.0766	15.21	NS (>400)	unstable	unstable
PSO	**0.0010**	**0.4147**	**0.0544**	**0.652**	**256.1**	**70.7**	**5.68**
GWO	0.0010	0.4136	0.0547	0.652	256.9	70.7	5.70
GA	0.0011	0.4164	0.0540	0.653	255.4	70.8	5.63
NSGA-II (knee)	0.0041	0.2885	0.1164	0.058	399.5	45.1	7.11

behaviour (Table 15) mirrored Case 1: the PSO design stayed below 0.7 % overshoot with settling times between 208.4 ms and 256.1 ms, whereas the Manual and PID Tuner designs failed to settle at 20 % load, and the PID Tuner design also showed 13.2 % overshoot and a settling time of 391.3 ms at $V_\mathrm{in} - 20\,\%$.

The optimizer statistics were consistent with Case 1 and more pronounced for GA (Table 12 and Figs. 10(b) and 9). PSO reached the best cost in 30 of 30 runs, GWO in 26 of 30 (86.7 %) and GA in only 1 of 30 (3.3 %), with a GA mean cost of 0.7458 and a worst cost of 0.9326. In contrast to Case 1, all pairwise differences were significant after the Bonferroni correction (Table 13; $p \leq 8.3 \times 10^{-6}$, including GWO against GA). GA solutions included K_p values up to 0.004, which the cost penalizes through a reduced gain margin and slower settling, indicating that the GA search did not consistently reach the corner of the feasible region where the optimum lies. The NSGA-II knee ($K_\mathrm{p} = 0.0041$, $K_\mathrm{i} = 0.2885$; Fig. 11(b)) reduced the worst-case overshoot to 0.058 % and kept $\mathrm{GM_{min}}$ at 7.11 dB, but its worst-case settling time reached 399.5 ms, essentially the entire simulation horizon, and $\mathrm{PM_{min}}$ fell to 45.1°; this solution should therefore be regarded as a conservative design rather than a practical one.

6.5 Cross-case robustness summary

Fig. 12 summarizes the worst-case overshoot, the worst-case settling time and the minimum phase margin of all controllers in both cases. The Manual, Calculated and PID Tuner designs are the same in both cases, so differences between the blue and red bars arise only from the optimized designs. Panel (a) shows that the optimized designs reduce the worst-case overshoot from 18.7 % (Manual)

Table 15: Overshoot (%) / settling time (ms) per scenario, Case 2. NS = not settled within 0.4 s.

Scenario	Manual	Calculated	PID Tuner	PSO	NSGA-II (knee)
Nominal	0.53 / 186.4	0.44 / 269.9	0.63 / 95.7	0.52 / 242.3	0.00 / 387.4
V_{in} − 20 %	1.16 / 169.3	0.63 / 241.8	13.24 / 391.3	0.64 / 208.4	0.06 / 355.0
V_{in} + 20 %	0.63 / 189.5	0.20 / 280.5	0.44 / 109.1	0.34 / 256.1	0.00 / 399.5
Load 50 %	0.98 / 221.5	0.11 / 271.7	0.38 / 113.2	0.21 / 246.8	0.00 / 384.1
Load 20 %	18.68 / NS	0.08 / 288.8	15.21 / NS	0.10 / 247.5	0.00 / 388.9
L − 20 %	0.53 / 189.9	0.42 / 274.0	0.54 / 101.4	0.51 / 247.0	0.00 / 391.4
C − 20 %	0.67 / 190.9	0.57 / 274.4	0.81 / 96.7	0.65 / 245.9	0.00 / 394.3

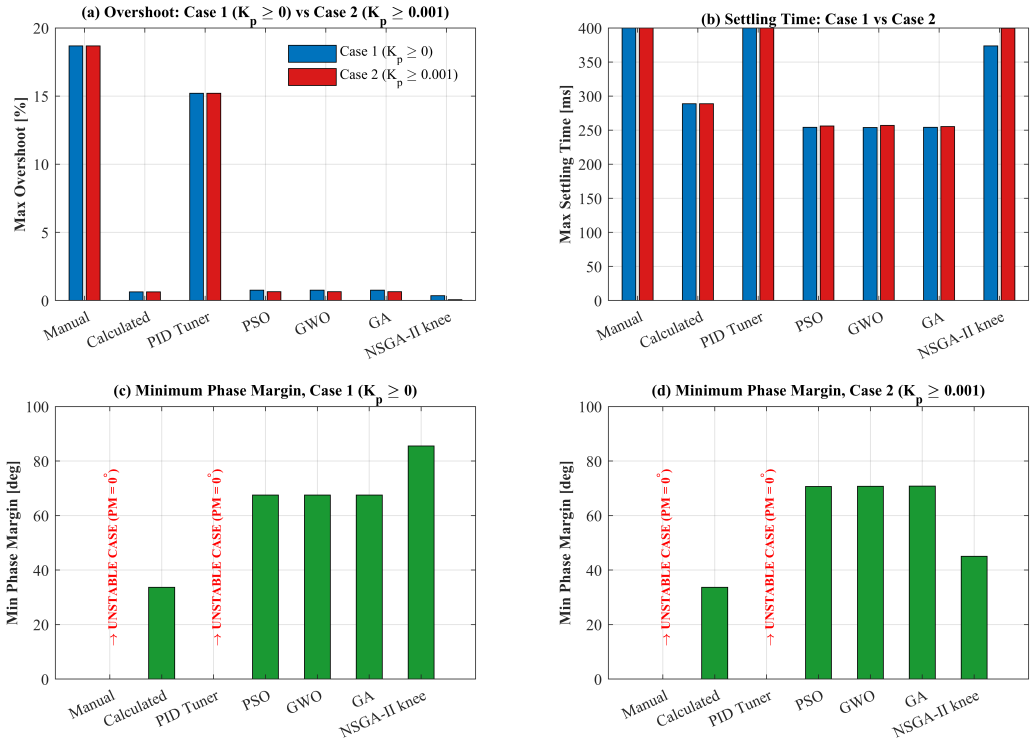


Figure 12: Cross-case robustness summary of all controllers: (a) worst-case overshoot, (b) worst-case settling time (bars reaching 400 ms denote designs that did not settle within the horizon), (c) minimum phase margin in Case 1 and (d) in Case 2 (designs marked unstable were unstable in at least one scenario of the averaged model; their phase margin is shown as 0°).

and 15.2 % (PID Tuner) to below 0.8 %, and that the lower bound on K_{p} slightly lowers the overshoot of the PSO, GWO and GA designs (from 0.76 % to 0.65 %). Panel (b) shows that the Manual and PID Tuner designs reach the 400 ms horizon (not settled), the Calculated design requires 289 ms, and the PSO, GWO and GA designs 254–257 ms; the NSGA-II knee design is the slowest (374 ms in Case 1 and 399.5 ms in Case 2). Panels (c) and (d) show that the Manual and PID Tuner designs are unstable in at least one scenario of the averaged model, whereas the minimum phase margin rises from 33.7° (Calculated) to about 67.5° in Case 1 and 70.7° in Case 2 for the optimized designs. Imposing $K_{\text{p}} \geq 0.001$ therefore changes the optimized designs only marginally (for PSO, +1.8 ms in settling time, −0.11 percentage points in overshoot and +3.2° in phase margin), which confirms that the robust optimum is insensitive to the proportional term as long as that term stays small.

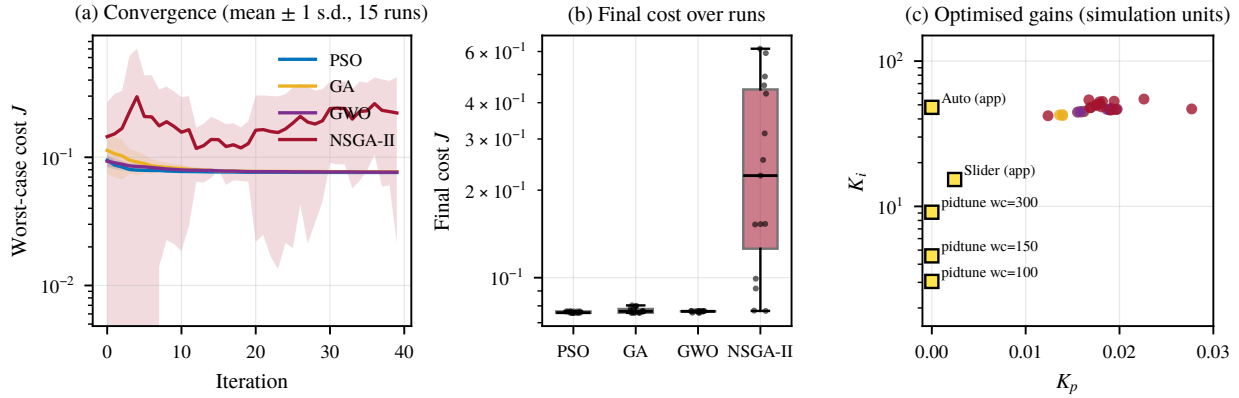


Figure 13: Study B, simulation: (a) convergence of the worst-case cost J (mean ± one standard deviation over 15 runs), (b) distribution of the final cost, (c) optimized gains of every run together with the reference designs (simulation units).

Table 16: Study B, simulation: statistics of the final worst-case cost J over 15 independent runs of each optimizer (equal budget of 680 cost evaluations per run). Success = runs within 0.5 % (2 %) of the best cost found by any optimizer. For NSGA-II the cost is that of the knee solution, which is not a minimizer of J.

Optimizer	Best	Median	Mean	Std	Worst	Within 0.5 %	Within 2 %	Time/run (s)
PSO	0.0757	0.0758	0.0761	0.0004	0.0766	8/15	15/15	81
GA	0.0757	0.0766	0.0771	0.0017	0.0803	6/15	11/15	72
GWO	0.0757	0.0766	0.0765	0.0004	0.0769	3/15	15/15	70
NSGA-II knee	0.0768	0.2245	0.2788	0.1913	0.6133	0/15	2/15	71

7 Results of Study B: Simulation of the Laboratory-Scale Converter

7.1 Optimizer performance and convergence

All four optimizers converged to the same region of the gain plane, $K_p \approx 0.016$–0.020 and $K_i \approx 45$–47 (Fig. 13(c)); the best costs of PSO, GA and GWO agree to within 0.04 % (0.07566, 0.07566 and 0.07569, Table 16), so the attainable solution quality does not depend on the choice among them. They differ in consistency. PSO reached a cost within 0.5 % of the best in 8 of 15 runs, GA in 6 and GWO in 3, and within 2 % in 15, 11 and 15 runs, respectively. PSO had the lowest mean cost (0.07610) with a standard deviation of 4.4×10^{-4}, GWO had the lowest standard deviation (3.8×10^{-4}, mean 0.07649), and GA had the widest spread (standard deviation 1.7×10^{-3}, worst run 0.0803, about 6 % above the best). PSO and GA needed on average 27 and 28 of the 39 iterations to come within 0.1 % of their final cost, and GWO 33. The rank-sum test (Table 17) indicates that PSO differs from GWO at the Bonferroni-corrected level ($p = 2.4 \times 10^{-4}$), whereas PSO against GA ($p = 0.019$) and GA against GWO ($p = 0.85$) do not. The NSGA-II knee is, by construction, not a minimizer of J, and its median cost (0.224) is three times higher; its Pareto front (hypervolume 0.79 ± 0.02) nevertheless contains solutions within 2 % of the single-objective optimum in only 2 of 15 runs. The differences among PSO, GA and GWO are small in absolute terms (below 2 % of the cost); PSO is the most consistent of the three, at a slightly longer run time (81 s against 70–72 s).

7.2 Worst-case performance on the averaged model

Table 18 gives the worst case over the 17 corner scenarios on the averaged model. The optimized designs of PSO, GA and GWO are practically identical ($K_p \approx 0.0161$–0.0162, $K_i \approx 45.0$): worst-case overshoot 0.31–0.32 %, worst-case settling time 4.0 ms, rise time 2.2 ms and minimum phase margin 65°. The NSGA-II knee trades 5 % longer settling (4.2 ms) and a lower margin (55.5°) for zero overshoot. The references are slower: 9.6 ms for the slider design of the PID Tuner app (2.4 times

Table 17: Study B, simulation: two-sided Wilcoxon rank-sum p-values of the final cost J (15 runs per optimizer). With six pairwise comparisons the Bonferroni-corrected level is $0.05/6 \approx 0.0083$.

Comparison	p-value	Significant
PSO vs GA	0.019	no
PSO vs GWO	2.4×10^{-4}	yes
PSO vs NSGA-II	3.1×10^{-6}	yes
GA vs GWO	0.852	no
GA vs NSGA-II	1.5×10^{-5}	yes
GWO vs NSGA-II	4.6×10^{-6}	yes

Table 18: Study B, simulation: worst case over the 17 corner scenarios, averaged (analytic) model. Reference designs come from `pidtune` and the PID Tuner app; the lower four rows are the optimizer designs (best of 15 runs).

Design	K_p	K_i	$\mathbf{OS}_{\max}$ (%)	$T_{s,\max}$ (ms)	$T_{r,\max}$ (ms)	$\mathbf{IAE}_{\max}$ (ms)	$\mathbf{PM}_{\min}$ (°)	J
pidtune wc=100	0.0000	3.04	0.00	46.9	26.3	12.03	87.9	0.9676
pidtune wc=150	0.0000	4.57	0.00	31.3	17.4	8.02	86.8	0.6449
pidtune wc=300	0.0000	9.12	0.00	15.6	8.6	4.02	83.6	0.3223
Auto (app)	0.0000	48.06	29.99	5.8	1.4	1.07	43.6	2.9403
Slider (app)	0.0024	15.31	0.00	9.6	5.2	2.39	84.8	0.1957
PSO	0.0162	45.02	0.32	4.0	2.2	0.81	64.9	0.0757
GA	0.0162	45.02	0.32	4.0	2.2	0.81	64.9	0.0757
GWO	0.0161	44.96	0.31	4.0	2.2	0.81	65.2	0.0757
NSGA-II knee	0.0196	46.36	0.00	4.2	2.2	0.79	55.5	0.0768

longer than the optimized designs), 15.6 ms for `pidtune` at 300 rad/s and 31–47 ms at 150 and 100 rad/s. The automatic design of the PID Tuner app is the fastest reference (5.8 ms) but has a worst-case overshoot of 30 % and a minimum phase margin of 43.6°, below the 45° target, which gives it by far the highest cost (2.94).

7.3 Monte Carlo robustness and disturbance rejection

In the 500 random held-out scenarios no design was unstable (Table 19). All optimized designs, the slider design and the `pidtune` design at 300 rad/s met the specification (overshoot ≤ 10 %, settling time ≤ 20 ms, phase margin $\geq 45°$) in 100 % of the scenarios, whereas the `pidtune` designs at 100 and 150 rad/s met it in none, because they are too slow (T_s above 30 ms at the 95th percentile), and the automatic PID Tuner design met it in 83.6 %, with overshoots up to 25.6 %. The 95th-percentile settling time was 4.0 ms for the optimized designs, 9.4 ms for the slider design and 15.2 ms for `pidtune` at 300 rad/s. The minimum phase margin over the Monte Carlo set was 69.7° for PSO and GA, 70.0° for GWO and 58.7° for the NSGA-II knee. For a -20 % step of the input voltage the optimized designs limited the peak output deviation to 15.2 % of V_{ref} at the nominal point and 18.7 % in the worst corner, against 19.7–24.1 % and 24.1–25.9 % for the references, and recovered within 2.4 ms against 4.2–27.7 ms. The load-current step (+20 %) produced a peak deviation below 0.27 % for all designs and is therefore not discriminating. The large deviations for the line step reflect the fact that the controller acts only through the duty ratio and has no feed-forward.

7.4 Switching-model validation

Table 20 and Fig. 15 give the switching-model results over 25 scenarios. The ranking by the Simulink worst-case cost is GWO (1.510), PSO (1.516), GA (1.516), NSGA-II knee (1.716), the automatic PID Tuner design (1.950), the slider design (2.156) and the three `pidtune` designs (2.16, 2.17 and 2.58). The optimized designs again have the shortest settling time (38.7–38.8 ms, NSGA-II 30.2 ms), the shortest rise time (2.6 ms) and the lowest steady-state error (about 1.1 mV), but the worst-case

Table 19: Study B, simulation: Monte Carlo robustness (500 random held-out scenarios; specification: overshoot $\leq 10\,\%$, $T_s \leq 20\,\text{ms}$, PM $\geq 45°$; no design was unstable) and disturbance rejection (peak deviation in percent of V_{ref} for a $-20\,\%$ input-voltage step and a $+20\,\%$ load-current step).

	Monte Carlo				Line step ($-20\,\%$)			Load step
Design	**Spec met (%)**	**OS 95th (%)**	**T_s 95th (ms)**	**PM_{min} (°)**	**Peak nom. (%)**	**Peak worst (%)**	**Recovery (ms)**	**Peak worst (%)**
pidtune wc=100	0.0	0.00	45.9	88.0	24.1	25.9	27.7	0.26
pidtune wc=150	0.0	0.00	30.5	87.0	24.0	25.8	18.4	0.26
pidtune wc=300	100.0	0.00	15.2	83.9	23.5	25.6	9.2	0.26
Auto (app)	83.6	17.99	4.0	47.8	19.7	24.1	4.2	0.27
Slider (app)	100.0	0.00	9.4	85.4	21.5	24.4	5.8	0.26
PSO	100.0	0.00	4.0	69.7	15.2	18.7	2.4	0.24
GA	100.0	0.00	4.0	69.7	15.2	18.7	2.4	0.24
GWO	100.0	0.00	4.0	70.0	15.3	18.7	2.4	0.24
NSGA-II knee	100.0	0.00	4.0	58.7	14.6	17.6	2.6	0.23

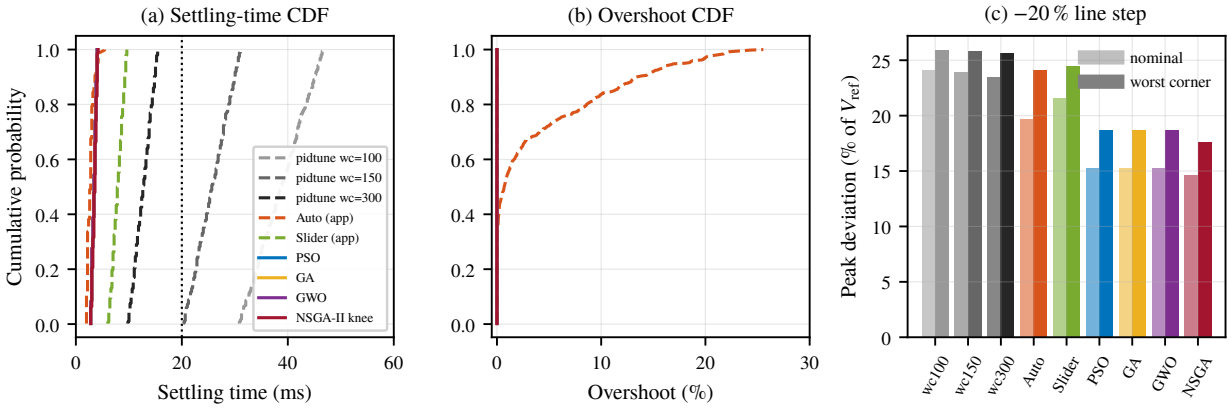


Figure 14: Study B, simulation, averaged model: (a) cumulative distribution of the settling time and (b) of the overshoot over 500 Monte Carlo scenarios (the dotted line in (a) is the 20 ms specification), (c) peak deviation for a $-20\,\%$ input-voltage step.

Table 20: Study B, simulation: switching-model (Simulink) validation over 25 scenarios (nominal, 15 worst corners, 9 random held-out). Steady error is the worst deviation of the mean output over the last 10 % of the run.

Design	K_p	K_i	**OS_{max} (%)**	**$T_{s,\text{max}}$ (ms)**	**$T_{r,\text{max}}$ (ms)**	**IAE_{max} (ms)**	**e_{ss} (mV)**	J_{sim}
pidtune wc=100	0.0000	3.04	8.88	92.7	30.74	12.03	136.38	2.1619
pidtune wc=150	0.0000	4.57	11.54	85.7	20.18	10.06	179.39	2.1700
pidtune wc=300	0.0000	9.12	15.60	90.3	10.76	7.92	138.51	2.5845
Auto (app)	0.0000	48.06	11.29	56.2	2.32	3.25	13.48	1.9497
Slider (app)	0.0024	15.31	14.25	70.5	6.78	5.65	58.67	2.1555
PSO	0.0162	45.02	14.29	38.7	2.64	2.16	1.12	1.5155
GA	0.0162	45.02	14.29	38.7	2.64	2.16	1.13	1.5156
GWO	0.0161	44.96	14.22	38.8	2.64	2.17	1.13	1.5097
NSGA-II knee	0.0196	46.36	17.19	30.2	2.84	2.01	0.94	1.7159

overshoot is not 0.3 % as in the averaged model but 14.2–14.3 % (17.2 % for the NSGA-II knee). The reference designs overshoot by 8.9–15.6 %. The three optimizer designs differ by less than 0.4 % in cost, which is within the numerical noise of the simulation, so the choice of optimizer is immaterial at this level; GWO is placed first only by 0.4 %.

The difference between the averaged and the switching results is shown in Fig. 16. In the switching model the output rises almost instantaneously to 75–95 % of the reference (the output capacitor is initially charged to the input voltage), whereas the averaged model starts from zero and follows a smooth response; the slower designs then show a pronounced dip before approaching the

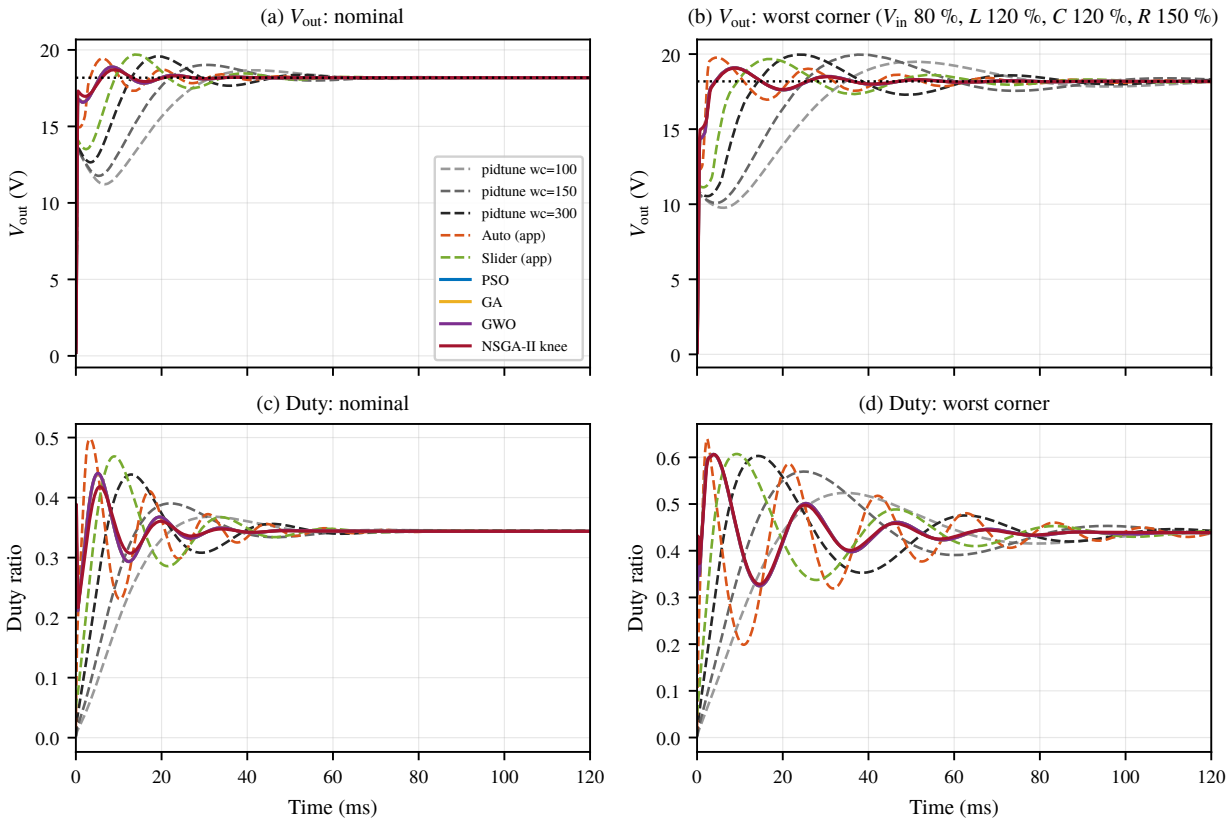


Figure 15: Study B, switching-model validation: output voltage and duty ratio of the nine designs for the nominal scenario and for the worst corner. The PSO, GA and GWO curves overlap.

reference. At the nominal point the PSO design has an overshoot of 3.95 % and a settling time of 16.1 ms in Simulink, against 0 % and 3.4 ms in the averaged model. The switching model also shows a steady duty ratio of about 0.34 against 0.45 assumed by the averaged model, which is consistent with discontinuous conduction (Section 2.6). The averaged model identified correctly that the optimized designs are better than the references (they take the first positions in both rankings), but it did not preserve the order of the references: the automatic PID Tuner design is the worst in the averaged ranking (cost 2.94) and the best reference in the switching ranking (1.95), whereas `pidtune` at 300 rad/s is the best `pidtune` design on the averaged model and the worst on the switching model.

8 Results of Study B: Hardware-in-the-Loop Optimization and Validation

8.1 Optimization on the converter

Every one of the 4320 cost evaluations (4 optimizers × 3 runs × 360) was a measurement on the converter. Fig. 17(a) shows the best measured cost found so far, averaged over the runs: PSO, GA and GWO decreased from 1.3, 2.0 and 1.5 to 0.64, 0.62 and 0.48 during the 59 iterations, while the NSGA-II knee cost decreased from 2.0 to 1.4. The final costs of the individual runs, 0.44–0.94 for PSO, GA and GWO, are however optimistic: the best design of every run is a minimum over many noisy measurements, and when the best design of each run was measured again its cost was higher in all nine runs of PSO, GA and GWO (and in one of the three NSGA-II runs); for example it was 3.00 instead of 0.81 for the second PSO run (Fig. 17(b), Table 21). The measurement noise of J is large: three consecutive evaluations of the same baseline gains ($K_P = 0.1$, $K_I = 20$) gave 1.87, 4.30 and 1.95. After re-evaluation the best costs were 0.60 (GA), 0.68 (GWO), 0.76 (PSO) and 1.28 (NSGA-II knee), the means 0.99 (GA), 1.23 (GWO), 1.68 (PSO) and 1.56 (NSGA-II), and

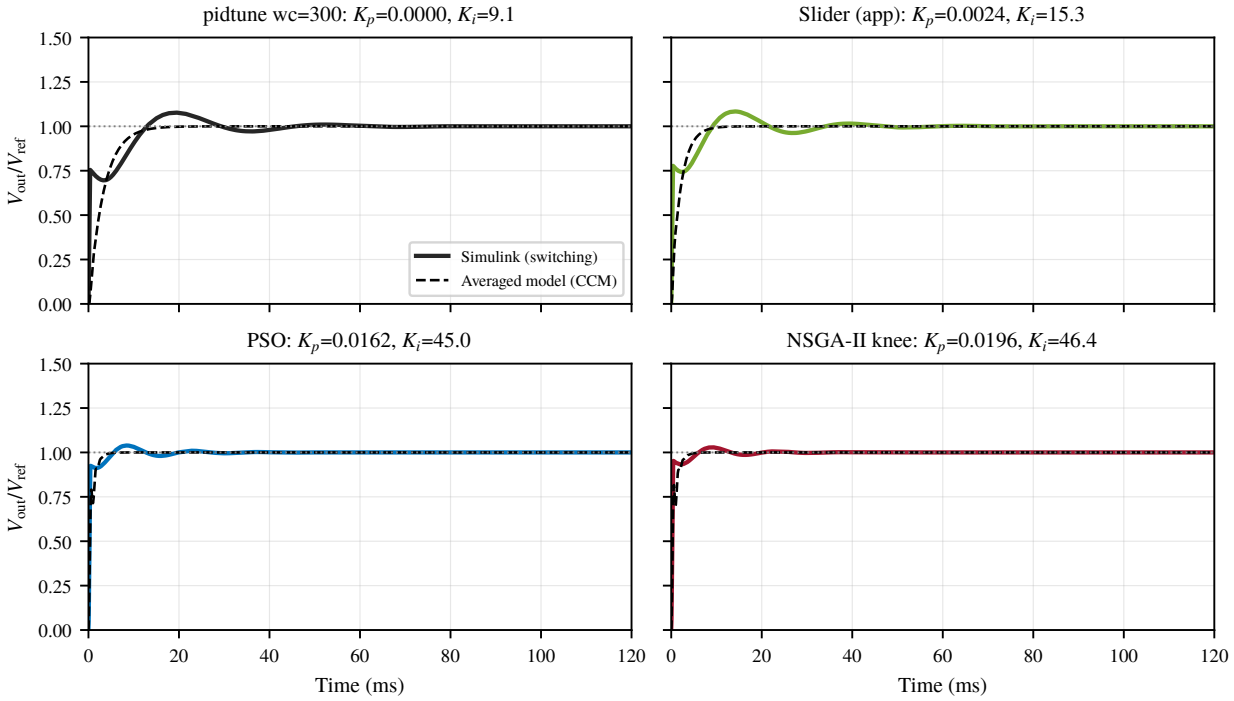


Figure 16: Study B, nominal start-up of four designs: switching model (Simulink, solid) versus averaged continuous-conduction model (dashed).

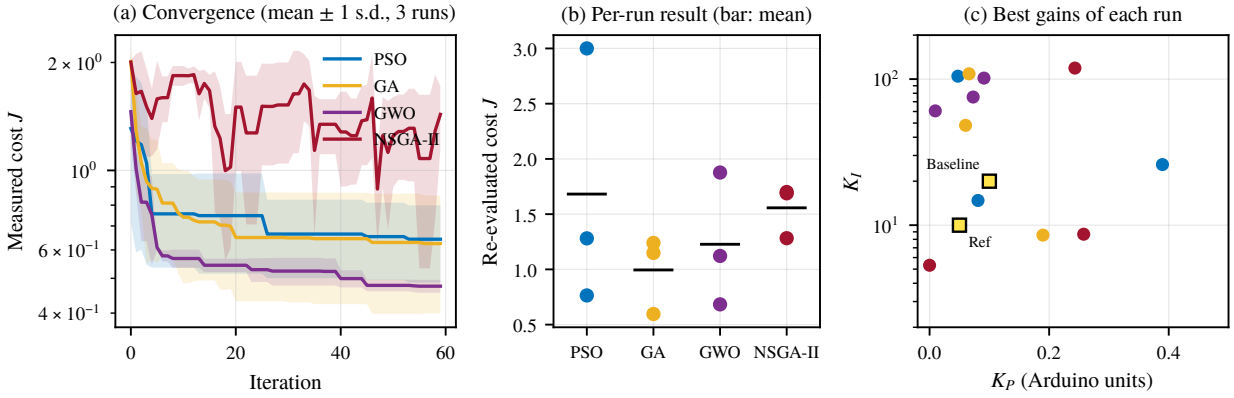


Figure 17: Hardware-in-the-loop optimization: (a) best measured cost found so far (mean ± one standard deviation over three runs), (b) re-evaluated cost of the best design of each run (bar: mean), (c) best gains of each run in Arduino units, with the baseline and reference gain pairs.

the standard deviations between 0.23 and 1.17. The p-values of Table 22 are all above 0.05 except one (GA against NSGA-II, 0.0495) that is based on only three runs per group and on a normal approximation; none of them supports a claim that one optimizer is better. The run optima are scattered (Fig. 17(c)), but five of the nine runs of PSO, GA and GWO ended at K_P between 0.047 and 0.091 and K_I between 48 and 109, a region that also contains the two best validated designs. The NSGA-II knees ($K_I = 5$–119) are more scattered and in one case give a very slow design.

Fig. 18 shows the measured cost of all evaluations in the gain plane. The median measured cost is 1.10 for the 824 evaluations with K_P between 0.03 and 0.1 and K_I between 40 and 110, against about 2.0 for $K_I < 40$, 1.54 for $K_I > 110$, and 1.95–2.18 for $K_P > 0.1$. The four simulation designs tested in Section efsec:transfer (stars) include two with $K_I \approx 280$–289, above the search range of the optimization.

Table 21: Hardware: statistics of the re-evaluated cost J of the best design of each run (3 runs per optimizer, 360 measured evaluations per run, two re-evaluations per run best). Gains are in Arduino units.

Optimizer	Best	Median	Mean	Std	Worst	K_P (best)	K_I (best)
PSO	0.764	1.281	1.682	1.171	3.000	0.0474	104.7
GA	0.596	1.149	0.995	0.348	1.239	0.0601	48.2
GWO	0.683	1.122	1.227	0.604	1.877	0.0729	75.4
NSGA-II knee	1.282	1.689	1.557	0.238	1.700	0.0000	5.3

Table 22: Hardware: Wilcoxon rank-sum p-values of the re-evaluated cost (3 runs per optimizer). With three runs the test has almost no power, so none of these values is used to claim a difference.

Comparison	p-value
PSO vs GA	0.275
PSO vs GWO	0.513
PSO vs NSGA-II	0.513
GA vs GWO	0.827
GA vs NSGA-II	0.050
GWO vs NSGA-II	0.513

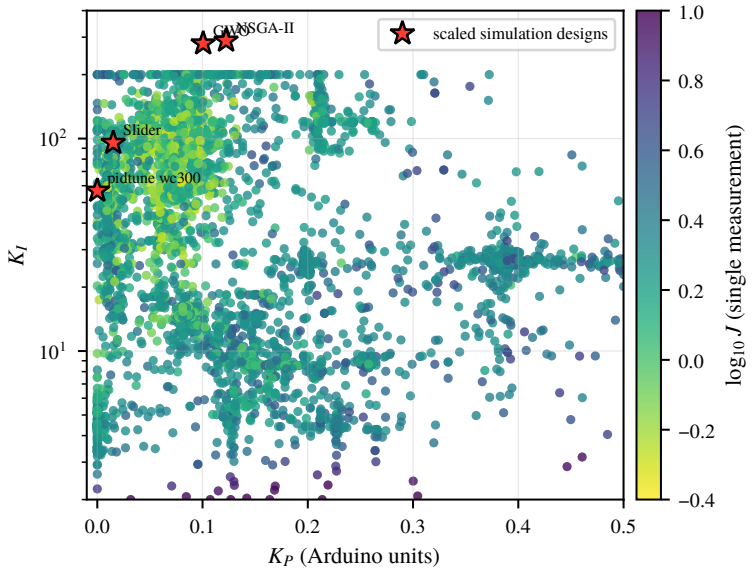


Figure 18: Measured cost of every hardware evaluation in the (K_P, K_I) plane (Arduino units) and the positions of the scaled simulation designs (stars).

8.2 Validation of the hardware-optimized designs

Table 23 and Figs. 19 and 20 give the validation results (four steps, three repeats). The designs found by PSO, GA and GWO are the fastest: their worst-case rise times are 16.2, 22.8 and 17.1 ms, against 72.0 ms for the baseline ($K_P = 0.1$, $K_I = 20$) and 130 ms for the reference ($K_P = 0.05$, $K_I = 10$), a three- to four-fold shorter rise time than the baseline, while the NSGA-II knee, with its low integral gain, is the slowest (225 ms). Their steady-state error is 1.0–1.5 % against 2.8 % for the baseline. The worst-case overshoot, 5.1–8.5 % (with repeat-to-repeat standard deviations of 1.4–5.6 percentage points) is of the same order as the 6.9 % of the baseline and 4.4 % of the reference, and it includes the output ripple of about 2 %; the overshoot is not reduced by the optimization. The validation cost is lowest for GA (0.88 ± 0.55), followed by PSO (1.25 ± 0.44), the NSGA-II knee (1.54 ± 0.12), GWO (1.71 ± 0.32), the reference (1.89 ± 0.06) and the baseline (2.87 ± 0.30); all four optimizer designs are 40–69 % below the baseline cost, but the differences among them are smaller

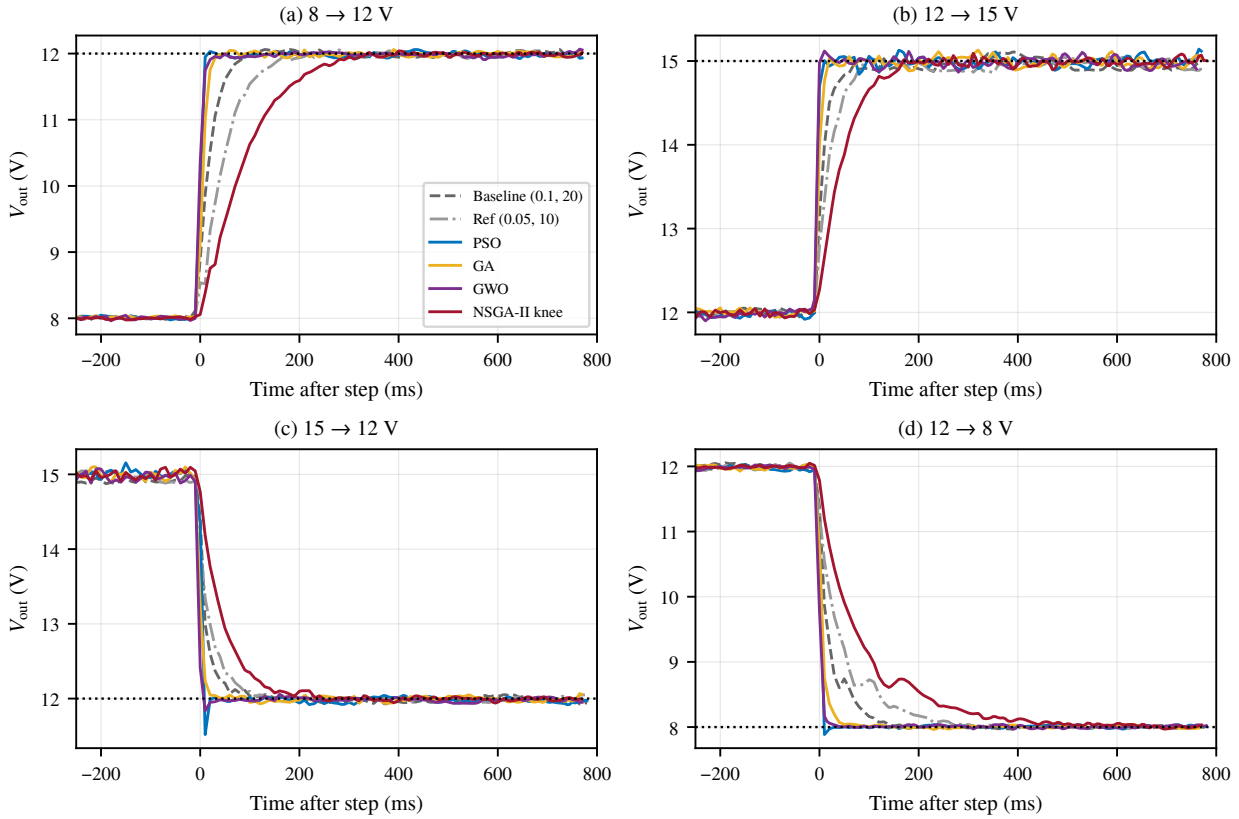


Figure 19: Measured step responses of the six validated designs (mean of three repeats): (a) 8→12 V, (b) 12→15 V, (c) 15→12 V, (d) 12→8 V.

Table 23: Hardware validation: worst case over the four steps 8 → 12 → 15 → 12 → 8 V, mean ± standard deviation of three repeats. Gains are in Arduino units. The first two rows are the reference designs measured on the bench; the other rows are the best design of each optimizer.

Design	K_P	K_I	**OS (%)**	T_r **(ms)**	T_s **(ms)**	e_{ss} **(%)**	**Ripple (%)**	J
Baseline (0.1, 20)	0.1000	20.0	6.9±0.3	72.0	743±26	2.83	1.64	2.873±0.304
Ref (0.05, 10)	0.0500	10.0	4.4±1.2	130.0	726±35	1.12	3.13	1.886±0.063
PSO	0.0474	104.7	8.5±5.6	16.2	278±428	1.49	1.99	1.251±0.439
GA	0.0601	48.2	5.1±1.4	22.8	284±415	1.02	2.04	0.883±0.549
GWO	0.0729	75.4	6.7±1.4	17.1	703±87	1.08	2.23	1.712±0.319
NSGA-II knee	0.0000	5.3	4.8±1.5	224.7	472±62	0.86	2.30	1.537±0.120

than one standard deviation of the repeats and the ranking must not be over-interpreted.

The settling-time column of Table 23 must be read with care. The worst-case T_s of PSO and GA is 278 ± 428 ms and 284 ± 415 ms, which means that in some repeats the output left the noise-aware band again late in the 0.8 s record, while in others it did not. The step-by-step results of the GA design (Table 25) show T_s of 35, 190, 770 and 45 ms for the four steps, although Fig. 19(c) shows that the output of this design has reached the new level within about 60 ms in the 15→12 V step; the long value is caused by excursions of the noisy output across the band at the small relative step height and not by a slow response. The rise time, which is not affected by this, is a more reliable speed indicator for the hardware. The ensemble-averaged metrics (Table 24) confirm the picture: rise times of 10–20 ms for the three optimizer designs, 72 ms for the baseline and 129 ms for the reference, and overshoots of 3.4–7.2 % for the optimized designs against 6.3 % for the baseline. A visible feature in Fig. 19(c) and (d) is the small undershoot of the fastest designs after a down-step (about 0.5 V for the PSO design in the 15→12 V step); a boost converter cannot sink current, so the

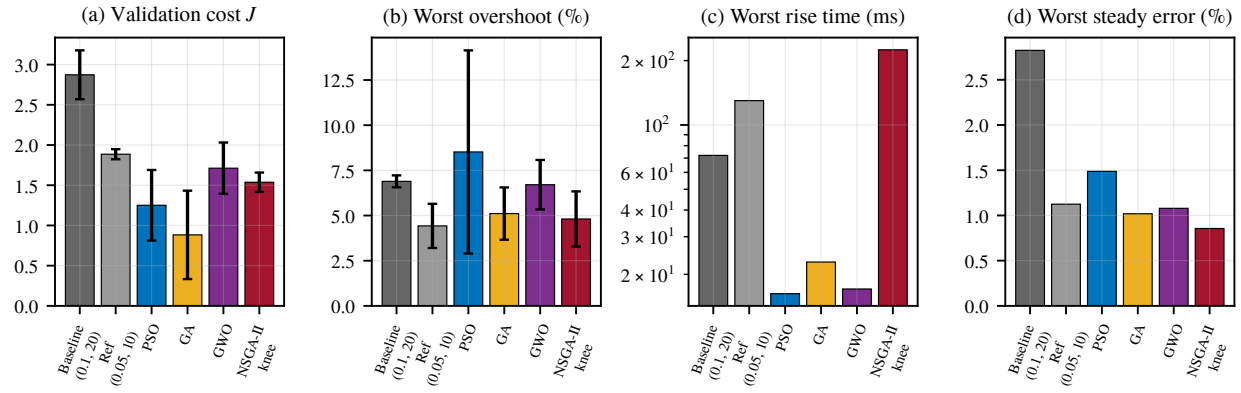


Figure 20: Hardware validation metrics of the six designs: (a) cost J (mean ± s.d.), (b) worst overshoot (mean ± s.d.), (c) worst rise time (logarithmic axis), (d) worst steady-state error.

Table 24: Hardware: ensemble-averaged metrics (the three repeats are averaged before the metrics are computed, which lowers the noise by $\sqrt{3}$), worst over the four steps.

Design	**OS (%)**	T_r **(ms)**	T_s **(ms)**	e_{ss} **(%)**
Baseline (0.1, 20)	6.29	72.1	780	2.83
Ref (0.05, 10)	1.99	128.7	770	0.50
PSO	7.15	10.0	770	1.02
GA	4.09	20.0	770	0.44
GWO	3.41	11.1	580	0.85
NSGA-II knee	2.66	228.4	430	0.61

Table 25: Hardware: step-by-step ensemble metrics of the design with the lowest validation cost (GA, $K_P = 0.0601$, $K_I = 48.24$).

Step	**OS (%)**	T_r **(ms)**	T_s **(ms)**	e_{ss} **(%)**	**Ripple (%)**
8→12 V V	1.22	19.0	35	0.12	0.69
12→15 V V	4.09	10.5	190	0.44	1.47
15→12 V V	1.86	11.6	770	0.23	0.93
12→8 V V	0.88	20.0	45	0.14	0.41

output falls only by discharging through the load and a fast integrator briefly cuts the duty to zero.

8.3 Selected hardware design

The design with the lowest validation cost is the GA solution $K_P = 0.0601$, $K_I = 48.24$. Its step-by-step metrics are given in Table 25: overshoot 0.9–4.1 %, rise time 10.5–20 ms and steady-state error 0.12–0.44 % over the four steps. Because the differences among the optimizer designs are within the noise, it is more honest to recommend the region than the single point: for this converter and this load, K_P between 0.045 and 0.075 and K_I between 48 and 105 gave rise times of 16–23 ms and a cost 40–69 % below the baseline.

9 Evaluation of the Simulation Gains on the Hardware

9.1 Selection and scaling

The best simulation design, selected by the switching-model cost (Table 20), is the GWO solution $K_p = 0.01614$, $K_i = 44.96$, which (17) converts to $K_P = 0.1006$ and $K_I = 280.2$. Three further simulation designs were scaled in the same way (Table 26): the NSGA-II knee ($K_P = 0.1224$,

Table 26: Bench re-test of the scaled simulation designs, derated versions of the best simulation design, and two control designs: worst case over the four steps $8 \to 12 \to 15 \to 12 \to 8$ V, mean (and standard deviation for J and OS) of five repeats measured in random order in one session. Gains in Arduino units. One rise-time value of zero (an artefact of the 10–90 % interpolation) was excluded from the mean T_r of the NSGA-II design.

Design	K_P	K_I	J	OS (%)	T_r (ms)	e_{ss} (%)	Ripple (%)	T_s (ms)
Simulation best (GWO), scaled	0.1006	280.2	3.85±1.20	29.6±11.8	16.2	2.56	4.32	584
NSGA-II knee, scaled	0.1224	288.9	2.89±0.56	12.8±5.4	15.8	1.26	5.55	737
PID Tuner slider, scaled	0.0151	95.4	1.83±0.50	10.3±3.4	14.4	1.63	2.84	749
`pidtune` 300 rad/s, scaled	0.0000	56.8	2.88±1.24	7.7±2.1	19.3	2.56	3.58	765
Simulation best, $K_i/3$	0.1006	93.4	2.87±1.29	9.7±2.1	16.8	2.66	3.99	751
Simulation best, $K_i/6$	0.1006	46.7	3.03±1.40	7.8±2.2	25.2	2.01	4.54	773
Hardware GA design (control)	0.0601	48.2	1.16±0.45	5.6±0.5	22.0	0.97	3.43	414
Baseline gains (control)	0.1000	20.0	4.19±1.43	10.5±1.2	54.2	3.64	3.05	767

$K_I = 288.9$), the `pidtune` design at 300 rad/s ($K_P = 0$, $K_I = 56.8$) and the slider design of the PID Tuner app ($K_P = 0.0151$, $K_I = 95.4$). Because PSO, GA and GWO produced practically identical simulation gains, one of them represents the group. In addition, two *derated* versions of the best simulation design were tested, with the integral gain divided by 3 ($K_I = 93.4$) and by 6 ($K_I = 46.7$) and the proportional gain unchanged. The hardware-tuned GA design and the baseline pair ($K_P = 0.1$, $K_I = 20$) were measured in the same session as controls. The baseline corresponds to $K_p = 0.016$, $K_i = 3.2$ in simulation units: the same proportional gain as the simulation optimum but a fourteen times smaller integral gain.

9.2 Bench measurement of the scaled designs

The eight designs were measured on the bench with the four-step sequence, five repeats each, in random order (Section 5.8). The result is given in Table 26 and Fig. 21. The scaled simulation-optimal design did not reproduce its simulated performance: its worst-case overshoot was 29.6 ± 11.8 % (against 4 % at the nominal start-up and 14.2 % in the worst of 25 switching scenarios), and its cost, 3.85 ± 1.20, was 3.3 times that of the hardware-tuned GA design (1.16 ± 0.45; Welch test $p = 0.005$, Mann–Whitney $p = 0.008$) and not distinguishable from that of the baseline gains (4.19 ± 1.43, $p = 0.70$). The NSGA-II knee, which has almost the same integral gain, had a cost of 2.89 ± 0.56 and an overshoot of 12.8 ± 5.4 %. The slider design, whose scaled integral gain is a third of the optimum, was the best of the scaled designs ($J = 1.83 \pm 0.50$, overshoot 10.3 ± 3.4 %), significantly lower than the best simulation design ($p = 0.016$) and not significantly different from the GA design ($p = 0.056$). The `pidtune` design had $J = 2.88 \pm 1.24$ and the lowest overshoot of the scaled designs (7.7 ± 2.1 %) but, being purely integral, a steady-state error of 2.6 %. An earlier, single measurement of the same four scaled designs gave the same order of cost (3.99, 2.07, 2.23 and 1.55 for GWO, NSGA-II, `pidtune` and slider) and overshoots of 20.9, 6.5, 3.9 and 6.1 %, so the failure of the best simulation design on the bench was observed twice.

Two cautions apply. First, the cost is noisy: its standard deviation over five repeats is 0.45–1.43, and the baseline gains gave 2.87 ± 0.30 in the validation (three repeats) but 4.19 ± 1.43 in this session, and 2.39, 2.40 and 4.19 in the three sanity tests that preceded it. Differences below about one unit of J between two designs are therefore not meaningful, and only the largest differences reach significance with five repeats. Second, the measured overshoot includes about 2 % of output ripple and the settling times of 410–770 ms are dominated by the noise-aware band (Section 8.2), so rise time, overshoot and cost are the more reliable indicators.

Derated designs. Lowering the integral gain of the best simulation design reduced the overshoot clearly, from 29.6 ± 11.8 % to 9.7 ± 2.1 % ($K_i/3$) and 7.8 ± 2.2 % ($K_i/6$), but did not lower the cost significantly (2.87 ± 1.29 and 3.03 ± 1.40 against 3.85 ± 1.20; $p = 0.25$ and 0.35), because the steady-state error (2.0–2.7 %) and the ripple (4.0–4.5 %) remained as large as for the non-derated design. A nearly controlled comparison is available: the $K_i/6$ design ($K_P = 0.101$, $K_I = 46.7$) and

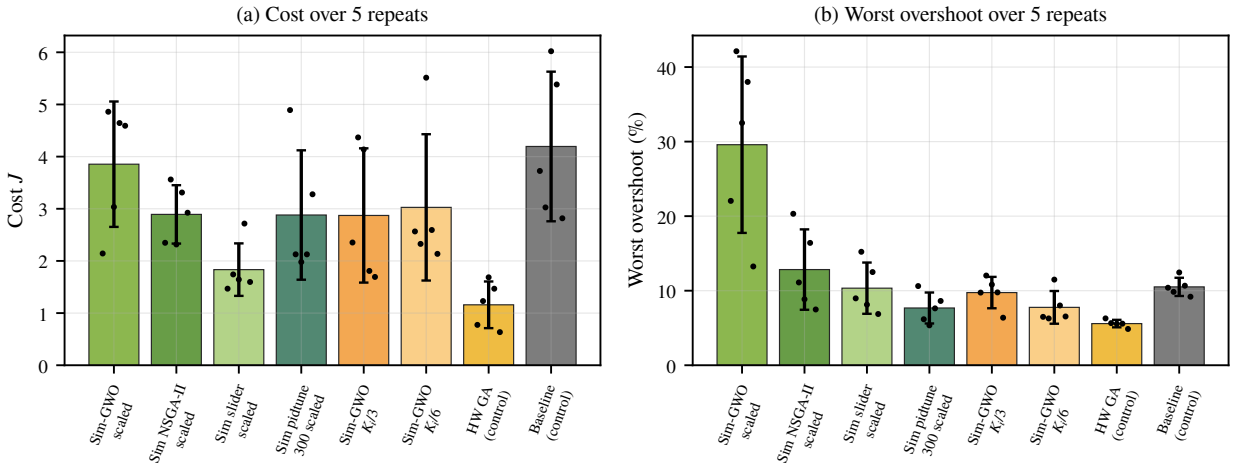


Figure 21: Bench re-test of the scaled simulation designs, the derated designs and the two controls: (a) cost J and (b) worst overshoot, mean ± standard deviation and the five individual repeats.

Table 27: Best hardware gains (Arduino units) converted back to simulation units by dividing by $S = 6.23$, and the ratio of the simulation optimum $K_i = 45.0$ to the converted integral gain.

Design	K_P	K_I	K_P/S	K_I/S	$45.0/(K_I/S)$
PSO	0.0474	104.7	0.0076	16.8	2.7
GA	0.0601	48.2	0.0096	7.7	5.8
GWO	0.0729	75.4	0.0117	12.1	3.7
NSGA-II knee	0.0000	5.3	0.0000	0.9	52.8
Baseline	0.1000	20.0	0.0160	3.2	14.0
Reference	0.0500	10.0	0.0080	1.6	28.0

the GA design ($K_P = 0.060$, $K_I = 48.2$) have almost the same integral gain and differ mainly in the proportional gain; the GA design had the lower cost (1.16 ± 0.45 against 3.03 ± 1.40, Welch $p = 0.038$, Mann–Whitney $p = 0.008$), a lower steady-state error (0.97 against 2.0 %) and a lower ripple (3.4 against 4.5 %). This suggests that the proportional gain, which passes the quantization noise of the ADC directly into the duty ratio, also has to be reduced, and that derating the integral gain alone is not sufficient. This was observed for one design family and has not been tested with a derated proportional gain.

9.3 Comparison with the hardware-optimized gains

Fig. 22 places the hardware optima in the simulation units by dividing them by S. The simulation optima of all 60 runs cluster at $K_p = 0.012$–0.028 and $K_i = 42$–55, whereas the best hardware designs of PSO, GA and GWO correspond to $K_i = 16.8$, 7.7 and 12.1 and $K_p = 0.0076$, 0.0096 and 0.0117 (Table 27). The integral gain that is best on the hardware is therefore 2.7 to 5.8 times lower than the simulation optimum, and the proportional gain is 0.5 to 0.7 times its value. The hardware optima straddle the slider and `pidtune` designs, which explains why the slider performed comparatively well after scaling, and they have both gains lower than the simulation optimum, which matches the observation of Section 9.2 that lowering only the integral gain was not sufficient.

9.4 Simulation with a sampled controller and the bench input voltage

The simulated controller of Section 4 is continuous, whereas the firmware is sampled at 1 kHz. Two additional switching-model simulations were made to see whether this difference, and the input voltage of 7.2 V, explain the gap (Section 5.8; Table 29).

Part A: start-up with a sampled controller. Replacing the continuous PI by a sampled PI with

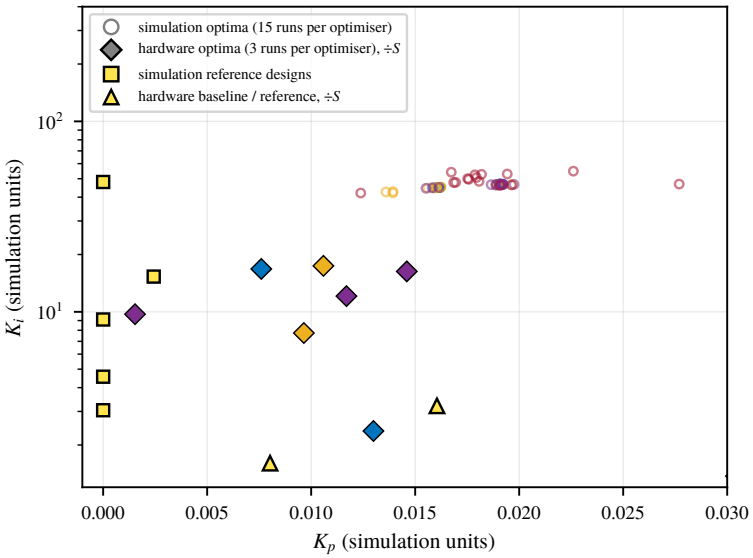


Figure 22: Optimized gains of the simulation (open circles) and of the hardware (diamonds, divided by S) in simulation units, with the reference designs.

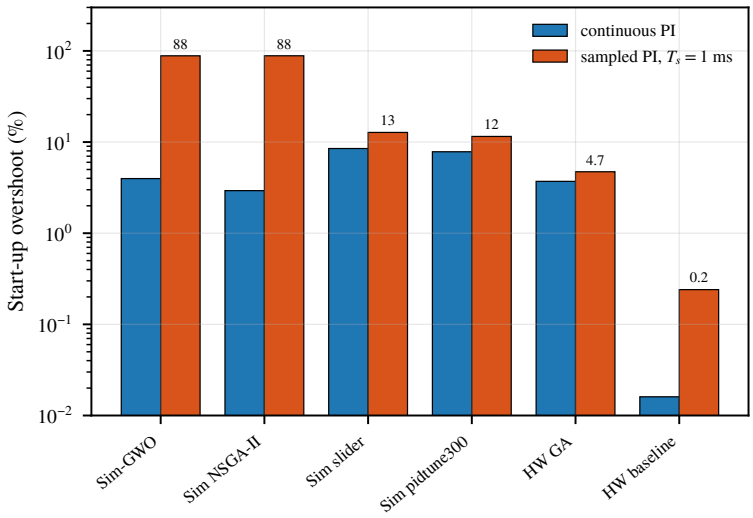


Figure 23: Start-up overshoot in the switching simulation with a continuous PI and with a sampled PI ($T_s = 1\,\text{ms}$), $V_\text{in} = 10\,\text{V}$.

$T_s = 1\,\text{ms}$ left the designs with small integral gain almost unchanged (slider 8.5 to 12.8 % overshoot, `pidtune` 7.8 to 11.5 %, hardware GA design 3.7 to 4.7 %, baseline 0.02 to 0.24 %; Fig. 23) but raised the overshoot of the two designs with $K_i \approx 45$ from 4.0 % and 2.9 % to 88.4 % and 88.5 % and their settling time from 16 and 11 ms to the end of the 120 ms record. The effect of sampling alone is thus large for the simulation optima and small for the designs that are good on the bench. This agrees with the first-order estimate of Table 28, in which the phase margin of the two simulation optima falls to 34–36° for a delay of one sample period while that of the hardware-tuned designs stays above 75°.

Part B: bench-like steps. With $V_\text{in} = 7.2\,\text{V}$ and the reference sequence of the bench, the sampled-controller simulation (Fig. 24) reproduces the main qualitative finding: the best simulation design has the highest cost (9.53) and the largest overshoot (98 %), followed by the NSGA-II knee (7.33, 74 %), and the ringing after each step lasts about 0.2 s, which is consistent with the long settling times measured on the bench for these designs. The simulated overshoot ranks the designs like the bench overshoot (Spearman $\rho = 0.57$ for the eight designs and $\rho = 0.89$, $p = 0.007$, without

Table 28: First-order estimate of the effect of the digital delay. Flat plant of gain $G_0 = 28\,\mathrm{V}$, PI controller in simulation units, pure delay τ: crossover ω_c and phase margin for $\tau = T_s/2$ and $\tau = T_s$ ($T_s = 1\,\mathrm{ms}$). The last column is the measured bench cost (mean of the five-repeat re-test where available, otherwise of the three-repeat validation; the two sessions differ, see Section 9.2). The estimate neglects the converter dynamics, quantization and sensor noise.

Design	K_i (sim. units)	ω_c (rad/s)	PM, $\tau = T_s/2$ (°)	PM, $\tau = T_s$ (°)	Bench J
Simulation best (GWO), scaled	45.0	1411	76	36	3.85
NSGA-II knee, scaled	46.4	1554	79	34	2.89
PID Tuner slider, scaled	15.3	430	82	69	1.83
`pidtune` 300 rad/s, scaled	9.1	255	83	75	2.88
Simulation best, $K_i/3$	15.0	470	103	90	2.87
Simulation best, $K_i/6$	7.5	235	110	103	3.03
Hardware GA	7.7	225	99	93	1.16
Hardware baseline	3.2	101	114	111	4.19
Hardware PSO	16.8	481	88	75	1.25
Hardware GWO	12.1	358	99	89	1.71
Hardware NSGA-II knee	0.9	24	89	89	1.54
Hardware reference	1.6	46	102	100	1.89

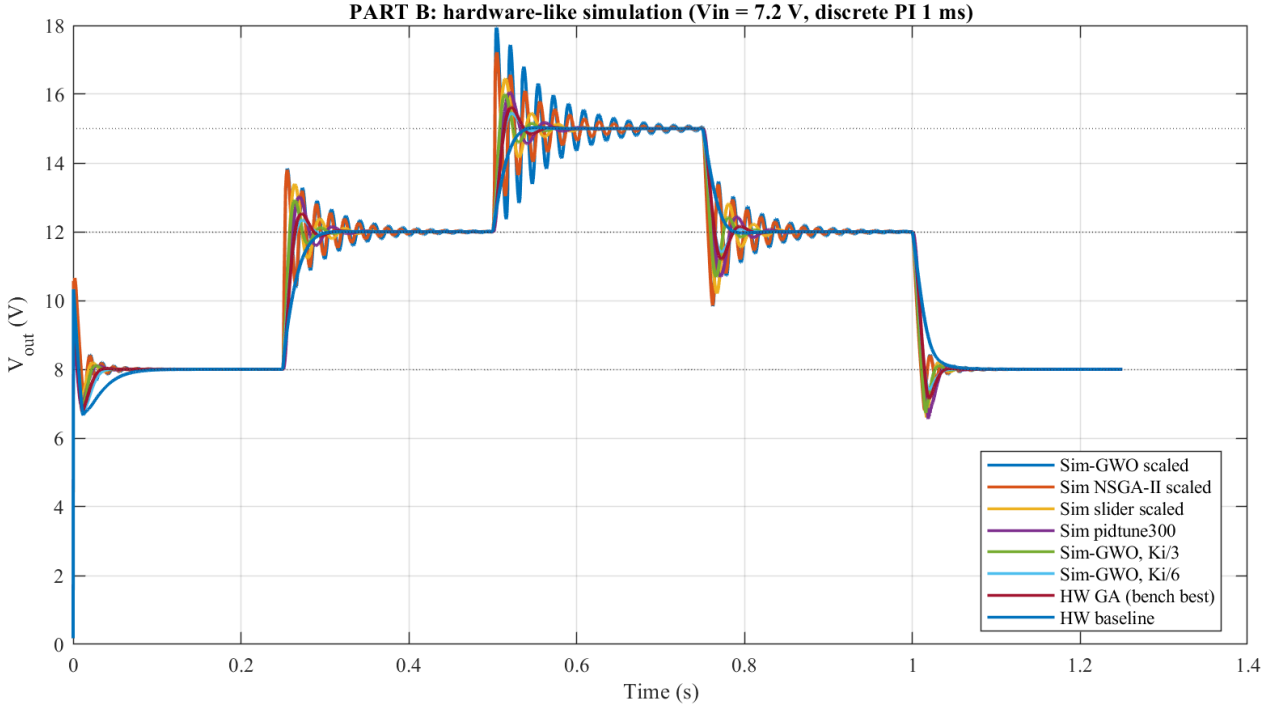


Figure 24: Simulation with a sampled PI ($T_s = 1\,\mathrm{ms}$) at $V_{\mathrm{in}} = 7.2\,\mathrm{V}$ and the bench reference sequence 8→12→15→12→8 V (0.25 s per level).

the baseline; Fig. 25(a)), and the simulation predicts, as the bench does, that the derated designs overshoot less (44 % and 20 % against 98 % in simulation; 9.7 % and 7.8 % against 29.6 % on the bench).

The simulation does not, however, reproduce the bench quantitatively. It overestimates the overshoot by a factor of 2.5 to 5.8 for the seven designs with a non-trivial overshoot (98 against 30 % for the best simulation design), it ranks the baseline gains first (cost 0.26) whereas the bench ranks them among the worst (4.19), and the correlation between simulated and measured cost is nil ($\rho = -0.10$; Fig. 25(b)). The cost of the bench contains the steady-state error and ripple penalties and noise-dominated settling times, which a noise-free simulation does not have, and the slow baseline suffers from exactly these terms. The bench input voltage therefore does not explain the gap on its own (the simulation at 7.2 V still shows large overshoots), and the sampled controller

Table 29: Simulation with a sampled PI controller (switching model, $T_s = 1\,\text{ms}$). Part A: start-up to 18.18 V from $V_{\text{in}} = 10\,\text{V}$, overshoot with a continuous and with a sampled controller. Part B: $V_{\text{in}} = 7.2\,\text{V}$, reference steps $8 \to 12 \to 15 \to 12 \to 8\,\text{V}$ (0.25 s each), worst case over the four steps, with the cost of (18) without noise penalties. The last two columns repeat the bench result of Table 26.

	Part A: OS (%)		**Part B: simulation**			**Bench**	
Design	**cont.**	**sampled**	**OS (%)**	T_s **(ms)**	J	**OS (%)**	J
Simulation best (GWO), scaled	4.0	88.4	98.1	243	9.53	29.6	3.85
NSGA-II knee, scaled	2.9	88.5	74.0	197	7.33	12.8	2.89
PID Tuner slider, scaled	8.5	12.8	59.8	96	5.76	10.3	1.83
`pidtune` 300 rad/s, scaled	7.8	11.5	43.5	87	4.28	7.7	2.88
Simulation best, $K_i/3$	–	–	43.7	63	4.20	9.7	2.87
Simulation best, $K_i/6$	–	–	19.7	52	1.97	7.8	3.03
Hardware GA design	3.7	4.7	26.5	55	2.63	5.6	1.16
Baseline gains	0.0	0.2	1.2	47	0.26	10.5	4.19

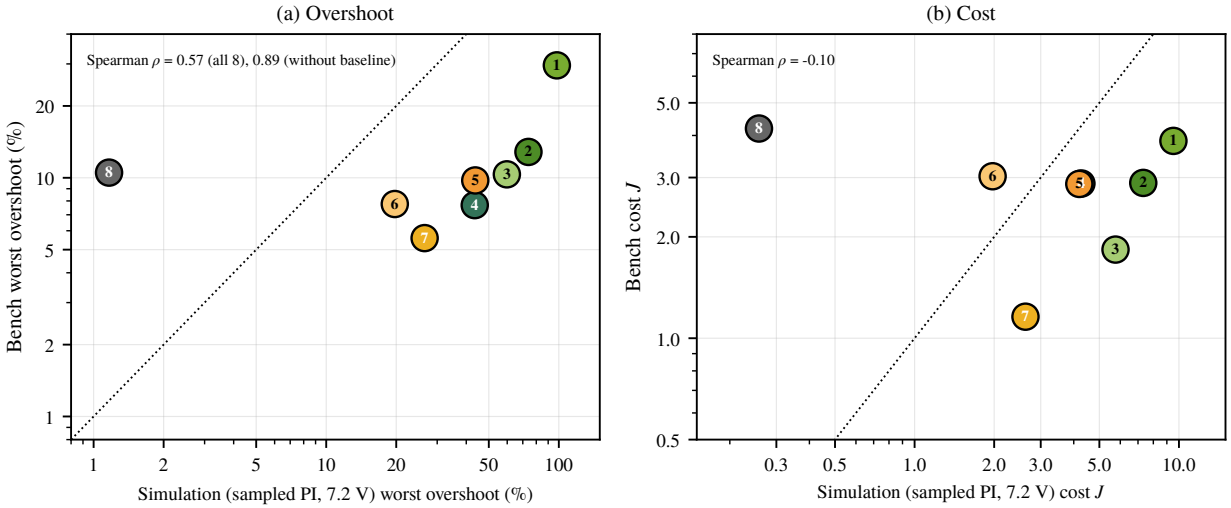


Figure 25: Simulation with a sampled PI at 7.2 V versus bench measurement for the eight designs of Table 26: (a) worst overshoot, (b) cost J. Numbers: 1 simulation best (GWO), 2 NSGA-II knee, 3 slider, 4 `pidtune` 300 rad/s, 5 and 6 simulation best with $K_i/3$ and $K_i/6$ (the marker of design 4 coincides with 5 in (b)), 7 hardware GA, 8 baseline. The dotted line is equality.

explains its direction better than its size.

9.5 Possible causes of the remaining gap

Digital delay. Supported by the sampled-controller simulation (Part A and B) and by the first-order estimate: the simulation optima have a crossover of about 1400–1550 rad/s, at which one sample period costs about 80° of phase. The effective delay of this firmware was not measured, and the simulation, which holds the duty for one period and has a one-period integrator lag, probably overestimates it, which would explain why the simulated overshoot is larger than the measured one.

Conduction mode. The converter operates in discontinuous conduction (Table 6), which the averaged CCM model used by the optimizers does not represent [11]. The switching model used in Part A and B includes it, and the estimated DC gain of the plant on the bench ranges from about 17 V (8 V level) to 29 V (15 V level), against 33 V in the averaged model.

Resolution and noise. One PWM count is 1/255 of the duty range, and with the ideal discontinuous-mode gain of about 27 V per unit duty at the 12 V level it changes the output

by about 0.1 V (0.9 % of 12 V, four ADC counts), which is of the same size as the observed ripple of about 2–5 %. A proportional gain, and to a lesser extent a high integral gain, converts the quantization and the ADC noise into duty activity. The simulation contains neither, which is the most likely reason why it ranks the slow baseline first while the bench does not, and why a lower proportional gain was better on the bench at equal integral gain.

Operating point. The bench supply of 7.2 V lies 28 % below the nominal 10 V of the simulation and outside the ±20 % range of the scenario set; the simulation at 7.2 V shows that this alone does not remove the gap.

These results suggest that the simulation cost for the tuning should include the sampling of the controller, and the ADC and PWM resolution, and that the scaled gains should be treated as an upper bound, to be reduced in both K_p and K_i; this has not been tested with a re-optimization on the sampled model and remains future work.

10 Discussion

10.1 What the simulation can and cannot deliver

The two studies show complementary results. In both, the metaheuristics converge to a small region of the gain plane and give designs that are better than the standard tools: in Study A the optimized design limits the worst-case overshoot to 0.76 % where the manual and automatic designs reach 18.7 and 15.2 %, and in Study B the optimized designs are about 2.4 times faster than the best reference with a minimum phase margin of 65° in the corner scenarios. In both, the choice among PSO, GA and GWO matters little for the quality of the final design and more for the consistency of the search. The simulation of Study B, however, did not predict the best hardware gains. Its optimum has a 3–6 times larger integral gain than the best gains found on the bench, and when scaled and applied unchanged it gave a cost that was 3.3 times that of the hardware-tuned design (3.85 ± 1.20 against 1.16 ± 0.45, five repeats, $p = 0.005$) and indistinguishable from that of the baseline gains. A simulation that includes the 1 ms sampling of the controller reproduces the direction of this failure (88 % start-up overshoot instead of 4 %) and ranks the overshoot of the bench designs correctly ($\rho = 0.89$ without the baseline), but it overestimates the overshoot by a factor of 2.5–5.8 and cannot reproduce the cost ranking, because it contains neither measurement noise nor the steady-state-error and ripple terms of the bench cost. The averaged model did correctly distinguish the optimized designs from the poorly tuned references, but it did not preserve the order among the references. The practical value of the simulation is therefore to provide a search region, a feasible starting point and a robust cost function, and, if the sampling is included, a screening of designs that are unlikely to work; it does not deliver the final gains. This agrees with the reality-gap literature in robotics [45, 46] and with the practice of tuning on the physical system [47, 48].

10.2 Cost and practicality of the platform

The hardware experiment used an Arduino Uno, a resistive divider, a driver module and a laptop running MATLAB. The control loop (1 kHz, 8-bit PWM at 62.5 kHz, 10-bit ADC) runs entirely on the microcontroller, and the serial link carries only gains, references and a 100 Hz telemetry stream. Despite this simple platform, 4320 closed-loop evaluations could be run unattended in 175 min with a safety abort, and the measured noise was low enough to distinguish designs whose rise times differ by a factor of three to ten. The platform does impose limits that a faster board would not: a 1 ms control period restricts the usable loop bandwidth, the quantization of the ADC and of the duty adds ripple of about 2 %, and settling-time metrics are hard to measure in the presence of this noise. These limits, rather than the price of the platform, are what the hardware-in-the-loop tuning had to work around, and they are part of the reason why the hardware optimum differs from the simulation optimum.

10.3 Study A: why the optimum is integral-dominant

The results are explained by the structure of the boost-converter power stage rather than by the choice of optimizer. The averaged plant (2) is flat at the DC gain V_{in}/D'^2 (48 at the nominal point) up to a resonance at f_0 and contains an RHP zero at f_z. For the converter studied here the two frequencies coincide at the nominal operating point ($f_z = f_0 = 31.8\,\mathrm{Hz}$, $Q = 1.0$), and the resonance becomes sharply underdamped at light load ($Q = 5.0$ at 20 % load, Table 4). Because the plant gain does not roll off before the resonance, a proportional term raises the loop gain at and above the resonance, where the phase is already close to $-180°$ (integrator lag plus resonant lag plus RHP-zero lag); it therefore consumes gain margin without providing useful bandwidth. Integral action, in contrast, sets a low crossover below the lowest RHP-zero frequency (20.4 Hz at V_{in} − 20 %), in line with the general rule that bandwidth must remain a fraction of the RHP-zero frequency [10]. The optimized gain $K_{\mathrm{i}} \approx 0.409$ corresponds to a crossover of approximately $K_{\mathrm{i}}G_0 \approx 19.6$ rad/s (3.1 Hz), about one tenth of the nominal RHP-zero frequency, which agrees with the analytical design rule of Section 3.7 and explains why the optimizer drove K_{p} to its lower bound.

The data support this interpretation. At almost equal K_{i} (0.405–0.415), increasing K_{p} from 0 (Case 1 PSO) to 0.001 (Case 2 PSO) to 0.004 (Calculated) lowered the minimum gain margin monotonically from 5.85 dB to 5.68 dB to 4.98 dB and lengthened the worst-case settling time from 254.3 ms to 256.1 ms to 288.8 ms. The robustness failures of the Manual and PID Tuner designs are explained by the same mechanism. This is visible in the loop-gain Bode diagrams of Fig. 26. The PID Tuner and the PSO (Case 1) designs are both pure integrators and therefore have identical phase; they differ only by a constant gain offset of $20\log_{10}(0.8837/0.4086) = 6.7$ dB. At 20 % load, the loop gain of the PID Tuner design reaches +0.85 dB at 31 Hz, where the phase is $-179°$: the loop gain exceeds unity at the $-180°$ crossing, so the loop is marginally unstable, in agreement with the 15 % overshoot and the absence of settling observed in the switching simulation (Fig. 8). The PSO (Case 1) loop reaches only −5.85 dB at the same frequency, which is exactly the minimum gain margin of 5.85 dB reported in Table 10. The Calculated design peaks at +0.86 dB, because its proportional term raises the loop gain at the resonance, but the proportional term also adds phase lead (the phase at the peak is $-121°$ instead of $-179°$), so the loop remains stable with reduced margins. Finally, a first-order estimate of the resonant loop gain for an integral controller, $|L(j\omega_0)| \approx (K_{\mathrm{i}}/\omega_0)\,G_0\,Q$, gives 0.49 for the PSO design and 1.06 for the PID Tuner design, in good agreement with the exact values of −5.85 dB (0.51) and +0.85 dB (1.10) obtained from (2).

10.4 Study A: robustness versus speed

The comparison exposes a clear trade-off. Tuning for the nominal point alone, as the PID Tuner does, gives the fastest nominal response (95.7 ms against 239.2 ms for PSO, i.e. about 2.5 times faster), but the resulting design is not robust: it shows 13–15 % overshoot in two of the seven scenarios and does not settle within the horizon at 20 % load. This conclusion applies to the default settings of `pidtune` (target phase margin of 60°) on the nominal plant; a more conservative target, or tuning on a worst-case plant, would give a more robust design, and the methodology of this paper is precisely a systematic way of obtaining such a design. The robust designs sacrifice nominal speed to guarantee acceptable behaviour everywhere, with a narrow settling-time band (204–254 ms) and overshoot below 0.8 %. The multi-objective results show that the trade-off can be shifted further toward robustness, as with the NSGA-II knee design, but only at the cost of settling times of 330–400 ms. The appropriate design depends on the application: where the load varies widely and start-up overshoot must be avoided, the robust PSO or Calculated designs are preferable; where operation is confined to the nominal point, a faster design could be justified provided that the operating range is enforced.

It should also be noted that the analytically derived design is itself a strong baseline. It was stable in all scenarios and had lower overshoot than the PSO design in all seven. The contribution of the optimization is therefore best described as a systematic refinement of an already sound analytical design (about 12 % faster settling, a doubling of the minimum phase margin and a small gain in

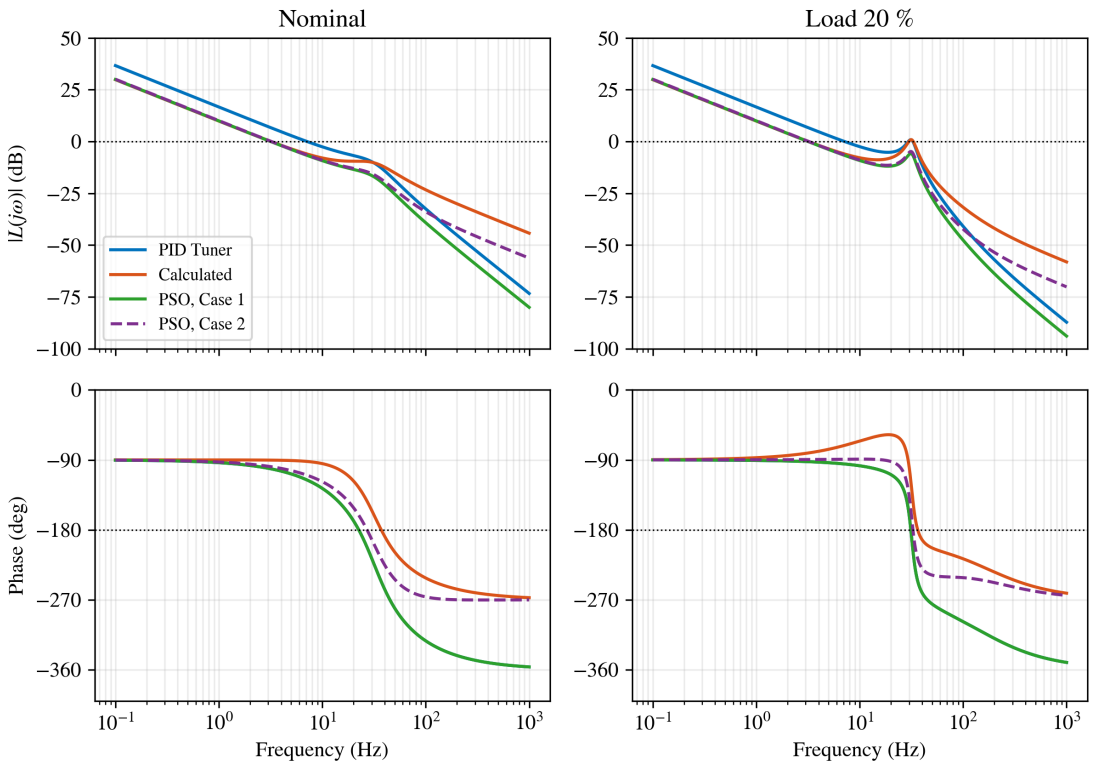


Figure 26: Loop-gain Bode diagrams of the averaged model for the nominal scenario and for 20 % load. The phase curves of the PID Tuner and PSO (Case 1) designs coincide because both are pure integrators.

gain margin), together with a demonstration of how fragile nominal-point designs are, rather than the discovery of a radically different controller.

10.5 Study A: behaviour of the optimizers

The three algorithms found the same best solution, which is expected for a two-parameter problem whose optimum lies on the boundary of a low-dimensional landscape. What differed was reliability. PSO converged to the optimum in every run in both cases, GWO did so in one third of the runs in Case 1 and in most runs in Case 2, and GA did so rarely. A plausible explanation is that the cost surface is non-smooth, with a steep penalty for unstable gains and a narrow, flat valley adjacent to the K_p bound; a population-based method with velocity memory such as PSO handles such a landscape well, whereas the default crossover and mutation operators of GA tend to produce candidates scattered across the bound. PSO and GWO converged in a similar number of iterations (7.5–8.6), so the advantage of PSO lies in the consistency of the final result rather than in speed. Reports on the relative merit of the algorithms differ: for a boost-converter PID problem, GWO gave a lower RMSE than PSO and GA in [24], whereas PSO was the most reliable here, which is consistent with the no-free-lunch argument that the ranking depends on the cost function, the search space, the budget and the algorithm settings. The established benchmark algorithms were used here to assess a tuning methodology, not to claim a new algorithm, in line with recent calls for rigorous benchmarking of metaheuristics [50].

These observations must be interpreted with caution. First, the PSO and GA populations were seeded with the three baseline designs, one of which (Calculated) lies within 1 % of the optimal K_i, whereas GWO started from random positions; this gives the seeded algorithms an advantage that was not isolated here. The fact that GA was seeded and still performed worst suggests that seeding does not explain the whole difference, but an unseeded comparison is needed to confirm this. Second, the population size and number of iterations were kept small (at most 240 evaluations) to make 30 repetitions affordable, and the GA operators were left at their default values; a tuned GA

could perform better. Third, the problem has only two decision variables, so the ranking should not be generalized to higher-dimensional controller structures, and no-free-lunch arguments [49] warn against generalizing across problems.

10.6 Positioning with respect to related work

The finding that metaheuristic tuning outperforms classical tuning is consistent with studies of PSO versus Ziegler–Nichols tuning in converter-based systems [2] and with work that reports robustness to input-voltage variations for an optimized controller on a non-minimum-phase boost model [28]. Experimental work with PSO-tuned PI gains on a boost-converter prototype [3] shows that such gains can be implemented in hardware, and online PSO auto-tuning of a PI controller has been reported for a dual active bridge converter [35]. The present study adds the following elements: the explicit use of a scenario set with worst-case aggregation and margin penalties; a comparison against nominal-point automatic tuners and an analytical design; repeated-run statistical comparison of several optimizers in simulation and on hardware; a documented gain-scaling rule to a low-cost microcontroller; and a quantified comparison between the simulation-optimal gains and the gains tuned on the converter. The implementation on a 1 kHz, 8-bit-PWM Arduino Uno complements work on low-cost platforms [36, 37, 40] by showing that such a platform can also serve as the measurement and optimization target. Learning-based controllers can improve step-response metrics relative to optimized PI control [14, 15], but they require training data and computing resources; the present results quantify how far a carefully tuned single-loop PI controller can be pushed, and where its structure limits it.

10.7 Practical design implications

Five practical guidelines follow. First, for a single-loop voltage-mode PI controller on a boost converter in continuous conduction, the controller should be integral-dominant, with a crossover of the order of one tenth of the lowest RHP-zero frequency (Study A). Second, tuning should consider worst-case operating points rather than the nominal point, which for the 500 W converter means light load combined with low input voltage. Third, the gains of a simulation should be converted to the units of the target with an explicit scaling rule such as (17), but the converted gains should be treated as an upper bound: on this converter the best gains found on the bench had both a 3–6 times lower integral gain and a 0.5–0.7 times lower proportional gain, and lowering only the integral gain of the simulation optimum reduced the overshoot (from 30 to 8–10 %) without reducing the cost significantly. Fourth, the simulation used for tuning should include the sampling of the controller, and ideally the conversion resolution and the conduction mode of the target, or the final tuning should be done on the converter. Fifth, a short hardware-in-the-loop tuning over a restricted search region, 360 measurements per run in the present case, is affordable on a low-cost platform and gave a cost 40–69 % below the baseline gains in the validation.

10.8 Limitations and threats to validity

The following limitations should be considered when interpreting the results.

1. The optimization of both studies used the averaged small-signal model, which assumes continuous conduction; Study B operates in discontinuous conduction (Table 6), so the absolute values of the averaged model do not match the switching model, and the optimization directly on the switching model or on a discontinuous-mode model might give different gains.
2. Study A is simulation-only and uses reference-step (start-up) responses; its worst scenarios were part of the optimization set, and simultaneous perturbations were not examined. Its settling times are capped by the 0.4 s horizon.
3. The simulation of Study B is not a digital twin of the bench: the simulated input voltage is 10 V against 7.2 V on the bench (outside the simulated uncertainty range), the simulated controller is

continuous, the simulated tests are start-ups to 18.2 V while the hardware tests are steps between 8 and 15 V, and the simulation includes no sensor noise or quantization. The comparison of overshoot and settling time between the two is therefore indicative only.

4. The hardware optimization used three runs per optimizer, a population of six and a noisy cost; the re-evaluation of the best designs showed that single measurements are optimistic. No statistically supported ranking of the optimizers can be given for the hardware, and the "best" hardware design (GA) is within the repeat-to-repeat variation of the others.

5. Settling times on the hardware depend on the noise-aware band and are not reliable at small step heights; the rise time and the cost are more robust indicators.

6. The cost weights and thresholds were chosen heuristically and influence the compromise between speed and robustness; only a single operating load of 50 Ω and one converter prototype were tested on the bench, and the scaled-gain test covers four designs and two derated variants, with five repeats measured in one session; the cost of the baseline gains differed between sessions (2.87 ± 0.30 and 4.19 ± 1.43), so absolute costs from different sessions should not be compared.

7. The effective delay of the firmware was not measured, and the sampled-controller simulation overestimates the overshoot of the bench by a factor of 2.5–5.8; the delay explanation of the simulation-to-hardware gap is therefore supported in direction, not in size. The derated designs were tested with the proportional gain unchanged, so the conclusion that the proportional gain also has to be reduced rests on one nearly controlled comparison.

11 Conclusions

This paper developed a robustness-aware tuning framework for the PI voltage controller of a DC–DC boost converter and evaluated it in simulation and on hardware, in a 500 W design-level study and in a laboratory-scale converter controlled by a basic Arduino Uno and a MATLAB serial link. The main conclusions are as follows.

1. In the 500 W study, controllers tuned manually or automatically at the nominal point were not robust (worst-case overshoot of 18.7 % and 15.2 %, no settling at 20 % load), whereas the PSO-tuned design kept the overshoot at or below 0.76 % with a worst-case settling time of 254 ms and roughly doubled the minimum phase margin of the analytical design. The optimum is integral-dominant because the RHP zero coincides with the LC resonance, and PSO reached the best cost in all 30 runs.

2. In the simulation of the laboratory-scale converter, PSO, GA and GWO converged to practically identical gains ($K_p \approx 0.016$, $K_i \approx 45$). Over 17 corner scenarios they gave a worst-case settling time of 4.0 ms on the averaged model, 100 % specification compliance over 500 Monte Carlo scenarios and the lowest switching-model cost, with PSO the most consistent of the three. The switching model nevertheless showed worst-case overshoots of 14 %, which the averaged continuous-conduction model did not predict because the converter operates in discontinuous conduction.

3. An explicit scaling rule, $K_{\mathrm{hw}} = S\,K_{\mathrm{sim}}$ with $S = 255 \cdot 25/1023 = 6.23$, converts simulation gains into the units of the Arduino firmware. Applied to the best simulation design it gave $K_P = 0.1006$ and $K_I = 280.2$, but on the bench (five repeats) this design produced a worst-case overshoot of 29.6 ± 11.8 % and a cost of 3.85 ± 1.20, 3.3 times that of the hardware-tuned GA design and not distinguishable from the baseline gains; an earlier measurement gave the same ordering. A switching simulation with the controller sampled at 1 ms raised the start-up overshoot of the simulation optima from 4 % to 88 % and ranked the bench overshoot correctly ($\rho = 0.89$ without the baseline), which supports the sampling delay as a main cause, but it overestimated the overshoot by 2.5–5.8 times and did not reproduce the cost ranking. Reducing only the integral gain of the simulation optimum lowered the overshoot to 8–10 % but not the cost.

4. Tuning directly on the converter, with 360 measured evaluations per run, gave designs with rise times of 16–23 ms against 72 ms for the baseline gains, steady-state errors of 1.0–1.5 % against 2.8 %, and validation costs 40–69 % below the baseline (0.88–1.71 against 2.87). The best hardware integral gains were 3–6 times lower, and the proportional gains 0.5–0.7 times lower, than the simulation optimum. With three runs per optimizer and a noisy cost, no optimizer can be declared superior on the hardware, and the hardware settling-time metric was limited by measurement noise.
5. The whole hardware study was carried out without high-cost real-time equipment such as dSPACE, OPAL-RT or STM32-based platforms: the control loop ran on an Arduino Uno (1 kHz, 8-bit PWM at 62.5 kHz, 10-bit ADC), and MATLAB, through its serial-communication interface, supplied the optimizers, the test sequence and the metrics. A low-cost set-up of this kind is therefore sufficient to tune a converter on the bench and to measure the simulation-to-hardware gap, although its sampling period and resolution limit the achievable bandwidth and add ripple.

For practice, the simulation should be used to define the search region and a robust cost, the scaled integral gain should be treated as an upper bound, and a short hardware-in-the-loop tuning should complete the design. Future work will include the sampling of the controller, the ADC and PWM resolution and the discontinuous conduction mode in the simulation cost, so that the scaled gains transfer directly, and will test a derating of both gains; measurement of the loop delay of the firmware; optimization directly on the switching model; additional runs and a larger population for the hardware optimization, with several load and input-voltage conditions; load-step and line-step tests on the bench; a cascaded voltage–current structure that is not limited by the RHP zero; and an unseeded, budget-matched comparison with additional metaheuristics.

Conflicts of Interest

The authors declare no conflicts of interest.

Declaration of Generative AI Use

During the preparation of this work the authors used Claude (Anthropic) for LaTeX formatting, language editing and consistency checking of the text, tables and references. The authors reviewed and edited the content and take full responsibility for it.

References

[1] R. W. Erickson and D. Maksimović, *Fundamentals of Power Electronics*, 3rd ed. Cham, Switzerland: Springer, 2020.

[2] K. Deželak, P. Bracinik, K. Sredenšek, and S. Seme, "Proportional-integral controllers performance of a grid-connected solar PV system with particle swarm optimization and Ziegler–Nichols tuning method," *Energies*, vol. 14, no. 9, p. 2516, 2021.

[3] L. D. Marin Uc, V. M. Ramirez Rivera, D. A. Uribe Sosa, and B. Saldivar, "Experimental validation of PI controller optimization using LPO and PSO for a DC–DC boost converter," *Sci*, vol. 8, no. 9, p. 250, 2026.

[4] A. Özdemir and Z. Erdem, "Double-loop PI controller design of the DC-DC boost converter with a proposed approach for calculation of the controller parameters," *Proc. Inst. Mech. Eng. I, J. Syst. Control Eng.*, vol. 232, pp. 137–148, 2018.

[5] K. J. Åström and T. Hägglund, *Advanced PID Control*. ISA – The Instrumentation, Systems, and Automation Society, 2006.

[6] J. G. Ziegler and N. B. Nichols, "Optimum settings for automatic controllers," *Trans. ASME*, vol. 64, pp. 759–768, 1942.

[7] K. J. Åström and T. Hägglund, *PID Controllers: Theory, Design, and Tuning*, 2nd ed. Research Triangle Park, NC, USA: Instrument Society of America, 1995.

[8] J. Berner, K. Soltesz, T. Hägglund, and K. J. Åström, "An experimental comparison of PID autotuners," *Control Eng. Pract.*, vol. 73, pp. 124–133, 2018.

[9] R. D. Middlebrook and S. Ćuk, "A general unified approach to modelling switching-converter power stages," in *Proc. IEEE Power Electron. Spec. Conf. (PESC)*, 1976, pp. 18–34.

[10] S. Skogestad and I. Postlethwaite, *Multivariable Feedback Control: Analysis and Design*, 2nd ed. Chichester, U.K.: Wiley, 2005.

[11] J. Sun, D. M. Mitchell, M. F. Greuel, P. T. Krein, and R. M. Bass, "Averaged modeling of PWM converters operating in discontinuous conduction mode," *IEEE Transactions on Power Electronics*, vol. 16, no. 4, pp. 482–492, 2001.

[12] R. B. Ridley, "A new, continuous-time model for current-mode control," *IEEE Trans. Power Electron.*, vol. 6, no. 2, pp. 271–280, Apr. 1991.

[13] H. Sira-Ramírez and R. Silva-Ortigoza, *Control Design Techniques in Power Electronics Devices*. London, U.K.: Springer, 2006.

[14] A. Marahatta, Y. Rajbhandari, A. Shrestha *et al.*, "Model predictive control of DC/DC boost converter with reinforcement learning," *Heliyon*, vol. 8, p. e11416, 2022.

[15] U. Saha, A. Jawad, S. Shahria, and A. B. M. Harun-Ur Rashid, "Proximal policy optimization-based reinforcement learning approach for DC-DC boost converter control: A comparative evaluation against traditional control techniques," *Heliyon*, vol. 10, no. 18, p. e37823, 2024.

[16] S. M. Ghamari, M. Hajihosseini, D. Habibi, and A. Aziz, "Design of an adaptive robust PI controller for DC/DC boost converter using reinforcement-learning technique and snake optimization algorithm," *IEEE Access*, vol. 12, pp. 141 814–141 829, 2024.

[17] A. Mármol, E. Zamiri, M. Purraji, D. Murillo, J. Tuñón Díaz, A. Vazquez, and A. de Castro, "Dual control strategy for non-minimum phase behavior mitigation in DC-DC boost converters using finite control set model predictive control and proportional–integral controllers," *Appl. Sci.*, vol. 14, no. 22, p. 10318, 2024.

[18] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. IEEE Int. Conf. Neural Netw. (ICNN)*, vol. 4, 1995, pp. 1942–1948.

[19] R. Poli, J. Kennedy, and T. Blackwell, "Particle swarm optimization," *Swarm Intell.*, vol. 1, no. 1, pp. 33–57, 2007.

[20] Z.-L. Gaing, "A particle swarm optimization approach for optimum design of PID controller in AVR system," *IEEE Trans. Energy Convers.*, vol. 19, no. 2, pp. 384–391, Jun. 2004.

[21] J. B. L. Fermeiro, J. Pombo, M. Calado, and S. Mariano, "A new controller for DC-DC converters based on particle swarm optimization," *Appl. Soft Comput.*, vol. 52, pp. 418–434, 2017.

[22] M. Thirumeni and D. Thangavelusamy, "Design and analysis of hybrid PSO–GSA tuned PI and SMC controller for DC–DC Ćuk converter," *IET Circuits Devices Syst.*, vol. 13, pp. 374–384, 2019.

[23] K. Premkumar, M. Vishnupriya, T. Thamizhselvan, P. Sanjeevikumar, and B. V. Manikandan, "PSO optimized PI controlled DC-DC buck converter-based proton-exchange membrane fuel cell emulator for testing of MPPT algorithm and battery charger controller," *Int. Trans. Electr. Energy Syst.*, vol. 31, p. e12754, 2021.

[24] J. Aguila-Leon, C. Chiñas-Palacios, C. Vargas-Salgado, E. Hurtado-Perez, and E. X. M. Garcia, "Particle swarm optimization, genetic algorithm and grey wolf optimizer algorithms performance comparative for a DC-DC boost converter PID controller," *Adv. Sci. Technol. Eng. Syst. J.*, vol. 6, no. 1, pp. 619–625, 2021.

[25] S. Mirjalili and A. Lewis, "The whale optimization algorithm," *Adv. Eng. Softw.*, vol. 95, pp. 51–67, 2016.

[26] B. Hekimoğlu, S. Ekinci, and S. Kaya, "Optimal PID controller design of DC-DC buck converter using whale optimization algorithm," in *Proc. Int. Conf. Artif. Intell. Data Process. (IDAP)*, 2018.

[27] A. Choubey, S. K. Jain, and P. K. Padhy, "A GWO-based indirect IMC-PID controller for DC-DC boost converter," *Energies*, vol. 17, no. 23, p. 5954, 2024.

[28] F. Merrikh-Bayat and A. Jamshidi, "Comparing the performance of optimal PID and optimal fractional-order PID controllers applied to the nonlinear boost converter," arXiv:1312.7517, 2013.

[29] A. Ambrisi, M. de Magistris, and R. Fresa, "Multi-objective optimization based design of high efficiency DC-DC switching converters," *Int. J. Power Electron. Drive Syst.*, vol. 7, no. 2, pp. 379–386, 2016.

[30] S. Mirjalili, S. M. Mirjalili, and A. Lewis, "Grey wolf optimizer," *Adv. Eng. Softw.*, vol. 69, pp. 46–61, 2014.

[31] J. H. Holland, *Adaptation in Natural and Artificial Systems.* Ann Arbor, MI, USA: Univ. Michigan Press, 1975.

[32] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, Apr. 2002.

[33] I. Boussaïd, J. Lepagnot, and P. Siarry, "A survey on optimization metaheuristics," *Inf. Sci.*, vol. 237, pp. 82–117, 2013.

[34] P. Palangpour *et al.*, "DSP-based PSO implementation for online optimization of power system stabilizers," in *Proc. NASA/ESA Conference on Adaptive Hardware and Systems*, 2008.

[35] S. Ab-Ghani *et al.*, "Adaptive online auto-tuning using particle swarm optimized PI controller with time-variant approach for high accuracy and speed in dual active bridge converter," *AIMS Electronics and Electrical Engineering*, 2023.

[36] J. A. González-Castro, G. J. Rubio-Astorga, M. A. Alarcón-Carbajal, J. D. Sánchez-Torres, M. Medina-Melendrez, J. C. Cabanillas-Noris, and D. E. Castro-Palazuelos, "Low-cost platform implementation of discrete controllers for DC-DC boost converter," *Energies*, vol. 17, no. 16, p. 4097, 2024.

[37] S. Nallusamy and D. K. Rukmani, "Design and simulation of Arduino Nano controlled DC-DC converters for low and medium power applications," *International Journal of Electrical and Computer Engineering*, vol. 13, no. 2, pp. 1400–1409, 2023.

[38] J. D. Vazquez-Plata, A. F. Lopez-Chavarro, E. A. Sanabria-Torres, J. F. Patarroyo-Montenegro, and F. Andrade, "Benchmarking real-time control platforms using a Matlab/Simulink coder with applications in the control of DC/AC switched power converters," *Energies*, vol. 15, no. 19, p. 6940, 2022.

[39] M. Metry, N. Guler, W. Rohouma, and R. S. Balog, "Performance comparison of dSPACE and MCU MPC implementation in PV power optimizer," in *Proc. IEEE Energy Conversion Congress and Exposition (ECCE)*, 2023, pp. 6266–6272.

[40] J. Aravena, D. Carrasco, M. Diaz, M. Uriarte, F. Rojas, R. Cardenas, and J. C. Travieso, "Design and implementation of a low-cost real-time control platform for power electronics applications," *Energies*, vol. 13, no. 6, p. 1527, 2020.

[41] A. Soriano, L. Marin, M. Valles, A. Valera, and P. Albertos, "Low cost platform for automatic control education based on open hardware," *IFAC Proceedings Volumes*, vol. 47, no. 3, pp. 9044–9050, 2014.

[42] M. S. Fadali, *Digital Control Engineering: Analysis and Design.* Academic Press, 2009.

[43] G. F. Franklin, J. D. Powell, and M. L. Workman, *Digital Control of Dynamic Systems*, 3rd ed. Addison-Wesley, 1998.

[44] P. Mattavelli, "Digital control of DC-DC boost converters with inductor current estimation," in *Proc. 19th Annual IEEE Applied Power Electronics Conference and Exposition (APEC)*, vol. 1, 2004, pp. 74–80.

[45] N. Jakobi, P. Husbands, and I. Harvey, "Noise and the reality gap: The use of simulation in evolutionary robotics," in *Advances in Artificial Life, ECAL 1995, Lecture Notes in Computer Science*, vol. 929. Springer, 1995, pp. 704–720.

[46] W. Zhao, J. P. Queralta, and T. Westerlund, "Sim-to-real transfer in deep reinforcement learning for robotics: a survey," in *Proc. IEEE Symposium Series on Computational Intelligence (SSCI)*, 2020, pp. 737–744.

[47] A. Marco, P. Hennig, J. Bohg, S. Schaal, and S. Trimpe, "Automatic LQR tuning based on Gaussian process global optimization," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2016, pp. 270–277.

[48] F. Berkenkamp, A. P. Schoellig, and A. Krause, "Safe controller optimization for quadrotors with Gaussian processes," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2016, pp. 491–496.

[49] D. H. Wolpert and W. G. Macready, "No free lunch theorems for optimization," *IEEE Trans. Evol. Comput.*, vol. 1, no. 1, pp. 67–82, Apr. 1997.

[50] K. Sörensen, "Metaheuristics—the metaphor exposed," *Int. Trans. Oper. Res.*, vol. 22, no. 1, pp. 3–18, 2015.

[51] E. van Dijk, J. N. Spruijt, D. M. O'Sullivan, and J. B. Klaassens, "PWM-switch modeling of DC-DC converters," *IEEE Trans. Power Electron.*, vol. 10, no. 6, pp. 659–665, Nov. 1995.

[52] D. Graham and R. C. Lathrop, "The synthesis of optimum transient response: Criteria and standard forms," *Trans. AIEE, Part II, Appl. Ind.*, vol. 72, no. 5, pp. 273–288, 1953.

[53] G. F. Franklin, J. D. Powell, and A. Emami-Naeini, *Feedback Control of Dynamic Systems*, 8th ed. New York, NY, USA: Pearson, 2019.

[54] Y. Shi and R. Eberhart, "A modified particle swarm optimizer," in *Proc. IEEE Int. Conf. Evol. Comput.*, 1998, pp. 69–73.

[55] The MathWorks, Inc., "Control system toolbox, global optimization toolbox, statistics and machine learning toolbox, simulink and simscape electrical documentation, R2024a," Natick, MA, USA. https://www.mathworks.com/help/, 2024, accessed: Oct. 2, 2026.

[56] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bull.*, vol. 1, no. 6, pp. 80–83, 1945.

[57] J. Derrac, S. García, D. Molina, and F. Herrera, "A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms," *Swarm Evol. Comput.*, vol. 1, no. 1, pp. 3–18, 2011.

[58] Microchip Technology, "ATmega328P 8-bit AVR microcontroller, datasheet," Manufacturer datasheet.